exported

November 4, 2023

0.1 The program below extracts the features from the 4 datasets: Balabit, Chaoshen, DFL and TWOS and teaches the models extracted behaviour.

The datasets consist of mouse actions

[ ]:
```
%pip install -r requirments.txt
```

```
Requirement already satisfied: asttokens==2.4.1 in .\.venv\lib\site-packages
(from -r requirments.txt (line 1)) (2.4.1)
Requirement already satisfied: colorama==0.4.6 in .\.venv\lib\site-packages
(from -r requirments.txt (line 2)) (0.4.6)
Requirement already satisfied: comm==0.1.4 in .\.venv\lib\site-packages (from -r
requirments.txt (line 3)) (0.1.4)
Requirement already satisfied: contourpy==1.2.0 in .\.venv\lib\site-packages
(from -r requirments.txt (line 4)) (1.2.0)
Requirement already satisfied: cycler==0.12.1 in .\.venv\lib\site-packages (from
-r requirments.txt (line 5)) (0.12.1)
Requirement already satisfied: debugpy==1.8.0 in .\.venv\lib\site-packages (from
-r requirments.txt (line 6)) (1.8.0)
Requirement already satisfied: decorator==5.1.1 in .\.venv\lib\site-packages
(from -r requirments.txt (line 7)) (5.1.1)
Requirement already satisfied: et-xmlfile==1.1.0 in .\.venv\lib\site-packages
(from -r requirments.txt (line 8)) (1.1.0)
Requirement already satisfied: exceptiongroup==1.1.3 in .\.venv\lib\site-
packages (from -r requirments.txt (line 9)) (1.1.3)
Requirement already satisfied: executing==2.0.1 in .\.venv\lib\site-packages
(from -r requirments.txt (line 10)) (2.0.1)
Requirement already satisfied: fonttools==4.44.0 in .\.venv\lib\site-packages
(from -r requirments.txt (line 11)) (4.44.0)
Requirement already satisfied: importlib-metadata==6.8.0 in .\.venv\lib\site-
packages (from -r requirments.txt (line 12)) (6.8.0)
Requirement already satisfied: importlib-resources==6.1.0 in .\.venv\lib\site-
packages (from -r requirments.txt (line 13)) (6.1.0)
Requirement already satisfied: ipykernel==6.26.0 in .\.venv\lib\site-packages
(from -r requirments.txt (line 14)) (6.26.0)
Requirement already satisfied: ipython==8.17.2 in .\.venv\lib\site-packages
(from -r requirments.txt (line 15)) (8.17.2)
Requirement already satisfied: jedi==0.19.1 in .\.venv\lib\site-packages (from
-r requirments.txt (line 16)) (0.19.1)
```

```
Requirement already satisfied: joblib==1.3.2 in .\.venv\lib\site-packages (from
-r requirments.txt (line 17)) (1.3.2)
Requirement already satisfied: jupyter_client==8.5.0 in .\.venv\lib\site-
packages (from -r requirments.txt (line 18)) (8.5.0)
Requirement already satisfied: jupyter_core==5.5.0 in .\.venv\lib\site-packages
(from -r requirments.txt (line 19)) (5.5.0)
Requirement already satisfied: kiwisolver==1.4.5 in .\.venv\lib\site-packages
(from -r requirments.txt (line 20)) (1.4.5)
Requirement already satisfied: matplotlib==3.8.1 in .\.venv\lib\site-packages
(from -r requirments.txt (line 21)) (3.8.1)
Requirement already satisfied: matplotlib-inline==0.1.6 in .\.venv\lib\site-
packages (from -r requirments.txt (line 22)) (0.1.6)
Requirement already satisfied: nest-asyncio==1.5.8 in .\.venv\lib\site-packages
(from -r requirments.txt (line 23)) (1.5.8)
Requirement already satisfied: numpy==1.26.1 in .\.venv\lib\site-packages (from
-r requirments.txt (line 24)) (1.26.1)
Requirement already satisfied: openpyxl==3.1.2 in .\.venv\lib\site-packages
(from -r requirments.txt (line 25)) (3.1.2)
Requirement already satisfied: packaging==23.2 in .\.venv\lib\site-packages
(from -r requirments.txt (line 26)) (23.2)
Requirement already satisfied: pandas==2.1.2 in .\.venv\lib\site-packages (from
-r requirments.txt (line 27)) (2.1.2)
Requirement already satisfied: parso==0.8.3 in .\.venv\lib\site-packages (from
-r requirments.txt (line 28)) (0.8.3)
Requirement already satisfied: Pillow==10.1.0 in .\.venv\lib\site-packages (from
-r requirments.txt (line 29)) (10.1.0)
Requirement already satisfied: platformdirs==3.11.0 in .\.venv\lib\site-packages
(from -r requirments.txt (line 30)) (3.11.0)
Requirement already satisfied: prompt-toolkit==3.0.39 in .\.venv\lib\site-
packages (from -r requirments.txt (line 31)) (3.0.39)
Requirement already satisfied: psutil==5.9.6 in .\.venv\lib\site-packages (from
-r requirments.txt (line 32)) (5.9.6)
Requirement already satisfied: pure-eval==0.2.2 in .\.venv\lib\site-packages
(from -r requirments.txt (line 33)) (0.2.2)
Requirement already satisfied: Pygments==2.16.1 in .\.venv\lib\site-packages
(from -r requirments.txt (line 34)) (2.16.1)
Requirement already satisfied: pyparsing==3.1.1 in .\.venv\lib\site-packages
(from -r requirments.txt (line 35)) (3.1.1)
Requirement already satisfied: python-dateutil==2.8.2 in .\.venv\lib\site-
packages (from -r requirments.txt (line 36)) (2.8.2)
Requirement already satisfied: pytz==2023.3.post1 in .\.venv\lib\site-packages
(from -r requirments.txt (line 37)) (2023.3.post1)
Requirement already satisfied: pywin32==306 in .\.venv\lib\site-packages (from
-r requirments.txt (line 38)) (306)
Requirement already satisfied: pyzmq==25.1.1 in .\.venv\lib\site-packages (from
-r requirments.txt (line 39)) (25.1.1)
Requirement already satisfied: scikit-learn==1.3.2 in .\.venv\lib\site-packages
(from -r requirments.txt (line 40)) (1.3.2)
```

```
Requirement already satisfied: scipy==1.11.3 in .\.venv\lib\site-packages (from
-r requirments.txt (line 41)) (1.11.3)
Requirement already satisfied: six==1.16.0 in .\.venv\lib\site-packages (from -r
requirments.txt (line 42)) (1.16.0)
Requirement already satisfied: stack-data==0.6.3 in .\.venv\lib\site-packages
(from -r requirments.txt (line 43)) (0.6.3)
Requirement already satisfied: threadpoolctl==3.2.0 in .\.venv\lib\site-packages
(from -r requirments.txt (line 44)) (3.2.0)
Requirement already satisfied: tornado==6.3.3 in .\.venv\lib\site-packages (from
-r requirments.txt (line 45)) (6.3.3)
Requirement already satisfied: traitlets==5.13.0 in .\.venv\lib\site-packages
(from -r requirments.txt (line 46)) (5.13.0)
Requirement already satisfied: typing_extensions==4.8.0 in .\.venv\lib\site-
packages (from -r requirments.txt (line 47)) (4.8.0)
Requirement already satisfied: tzdata==2023.3 in .\.venv\lib\site-packages (from
-r requirments.txt (line 48)) (2023.3)
Requirement already satisfied: wcwidth==0.2.9 in .\.venv\lib\site-packages (from
-r requirments.txt (line 49)) (0.2.9)
Requirement already satisfied: zipp==3.17.0 in .\.venv\lib\site-packages (from
-r requirments.txt (line 50)) (3.17.0)
Note: you may need to restart the kernel to use updated packages.
```

[ ]:
```
## IMPORTING THE CLASSES ##
from utils.consts import * ## const variables are there, so paths and␣
↪extraction features ##
from utils. plotting import * ## plotting ##
from data_reader_babalit import * ## the class which reads balabit dataset ##
from data_reader_chaoshen import * ## the class which reads chaoshen datasets␣
↪TODO There are several issues with this dataset -> explained later ##
from data_reader_singapur import * ## the class which reads singapur dataset##
from data_reader_dfl import * ## the class which reads dfl dataset ##

from nonlegality_analyser import * ## the class which analyses the datasets␣
↪without 0 1 labels --> splitting samples is being done there ##
from legality_analyser import * ## the class which analyses the datasets with 0␣
↪1 labels --> splitting samples is being done there ##


from SVMmodel import *
from RandomForestModel import *
from OneClassSVMModel import *
from IsolationForestModel import *

import random
```

1 BALABIT DATASET FEATURE EXTRACTION

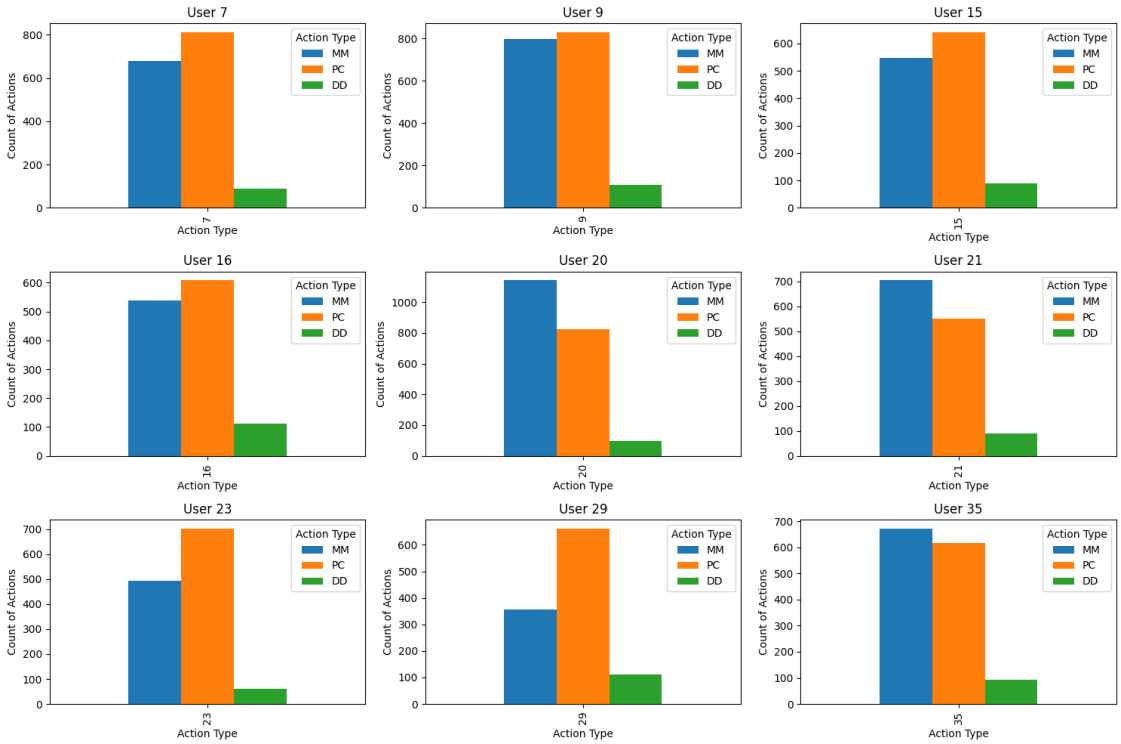
[ ]:
```
## Specyfing the USERS ##
users = [7,9,15,16,20,21,23,29, 35] ## those are all users
## Creating the object which analyses the dataset ##
balabit_reader = DataReaderBalabit(BALABIT, users, False, limit = 1000) ## the␣
↪arguments are DATASET, users, supervised, how many records should be␣
↪procesed ##
balabit_reader.processDataWithoutLabels() ## creating the data without labels ##
```

```
File already exist
```

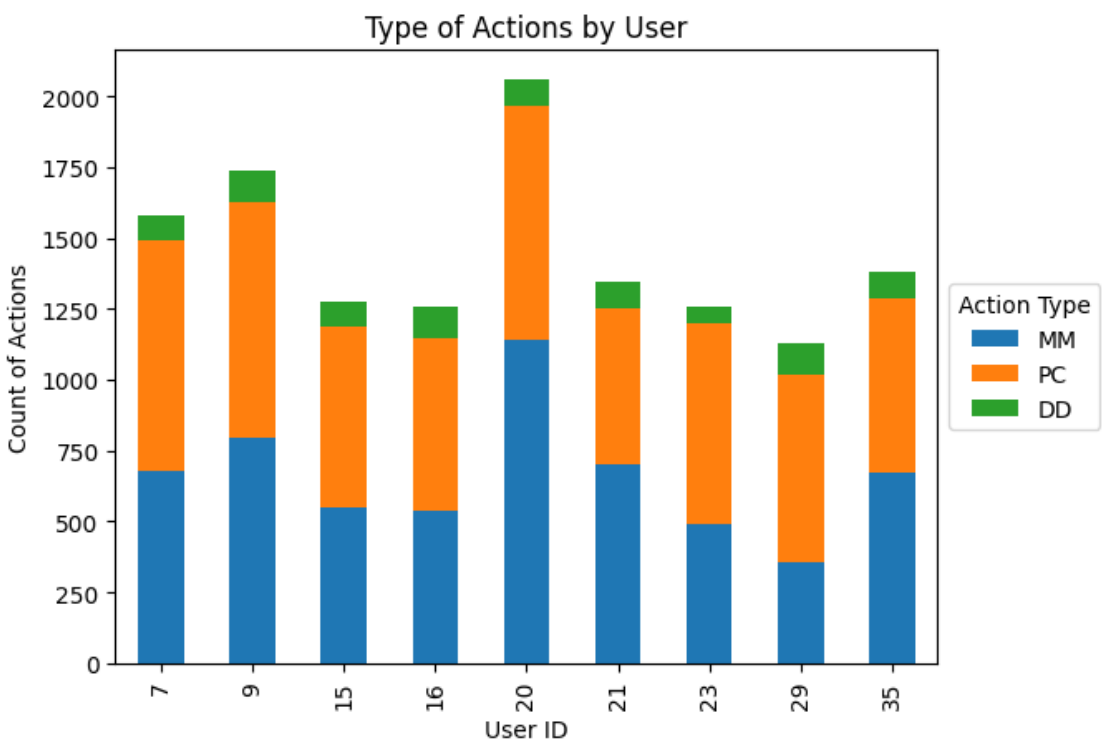
[ ]:
```
## getting path from the reader ##
print(balabit_reader.fileName)
path = balabit_reader.getFileName()

## creating analyser class ##
balabitAnalyser = nonLegalityAnalyser(path)
## counting how many actions were extracted ##
print(balabitAnalyser.countActions())
```

```
processed_files\balabit_dataset_users[7, 9, 15, 16, 20, 21, 23, 29,
35]_limit1000_labelsFalse.csv
type_of_action     1    3    4
userid
7                679  812   88
9                798  830  107
15               547  641   89
16               538  608  113
20              1141  823   95
21               704  550   90
23               494  703   61
29               357  662  112
35               672  617   94
```

[ ]:
```
## histogram of actions ##
balabitAnalyser.plotActionHistograms()
```




```
[ ]: balabitAnalyser.plotTypeOfActions(True)
```



The actions are represented by 3 variables: - MM (Mouse Move action) the user moves the cursor from x1,y1 to x2,y2 point, - PC (Process and Click action) the user moves the cursor from x1,y1 point to x2,y2 point and then proceeds to click the mouse, - DD (Drag and drop action) the user drags something from first point to another one.

2 BALABIT TEACHING THE MODELS

2.1 Supervised part:

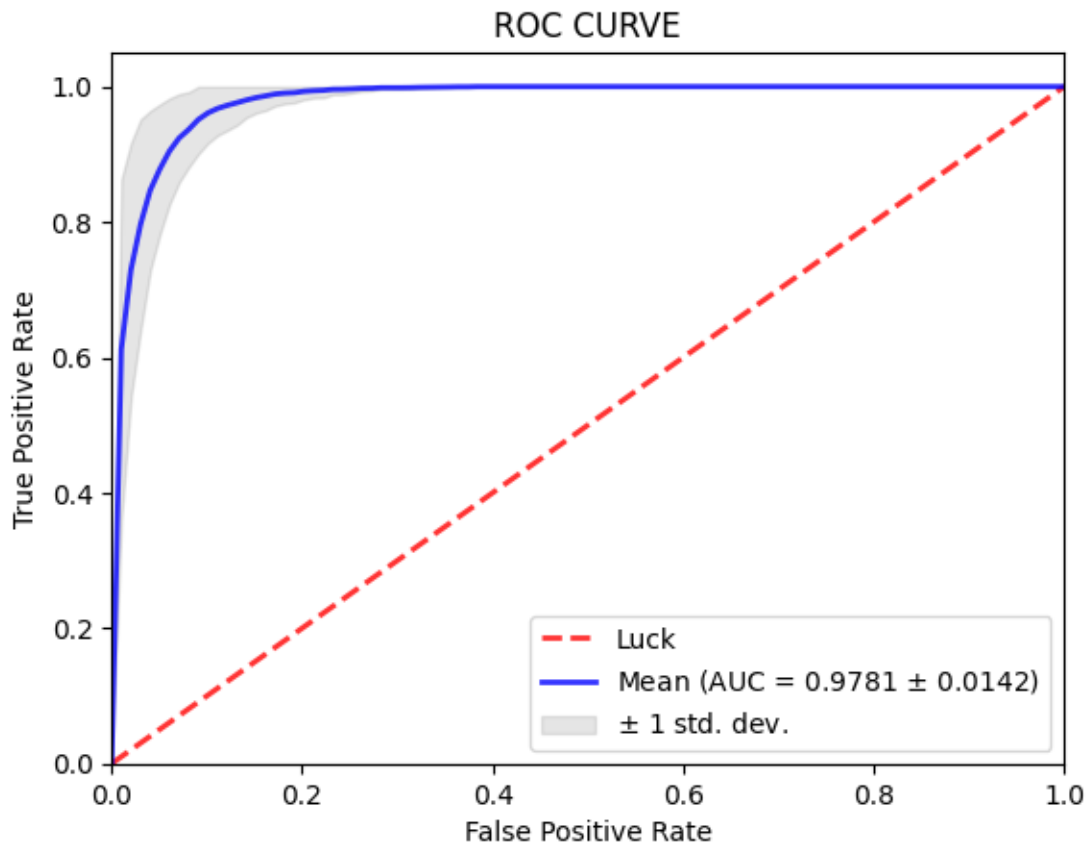
RandomForest and SVM

[ ]:
```
from RandomForestModel import *
```

[ ]:
```
# shuffle var will be used later in order to mix the data ##
shuffle = True
## dictionaries of fpr tpr ##
fpr = {}
tpr = {}
roc_auc = {}

# ONE USER IS ALWAYS LEGAL ##
for legalUser in users:
    ## this function gets all samples of legaluser and then randomly selects␣
↪the samples from other users - The data will be balanced ##
    dataset = balabitAnalyser.createTrainingDataWithLabel(legalUser)
    ## the last label is 1 or -1
    X = dataset[:, 0:-1]
    y = dataset[:, -1]
    ## train test split --> shuffle is used there ## if not used then data will␣
↪be divided as x[0:halfTrainSamples] concatenate with X[positiveSamples:␣
↪positiveSamples + trainHalfSamples]
    ## where positiveSamples is len(X/2)

    X_train, X_validation, y_train, y_validation = balabitAnalyser.
↪trainingTestSplit(X, y, TEST_SIZE, shuffle)
    model = RandomForestModel(dataset, users)
    ## evaluating the model, the variables are X_train, y_train, X_validation,␣
↪y_validation, scale --> use standardScler, and legalUser
    fpr[legalUser], tpr[legalUser], thr = model.evaluate(X_train, y_train,␣
↪X_validation, y_validation, scale = True, user = legalUser)
    threshold = -1
    try:
        eer = brentq(lambda x: 1. - x - interp1d(fpr[legalUser],␣
↪tpr[legalUser])(x), 0., 1.) ## Equal Error Rate //brentq znajdowanie 0 ,␣
↪interpid interpolacja
```

```
        threshold = interp1d(fpr[legalUser], thr)(eer)

    except (ZeroDivisionError, ValueError):
        print("Division by zero")

    roc_auc[legalUser] = auc(fpr[legalUser], tpr[legalUser])
    print(f"{legalUser}: {roc_auc[legalUser]:.3f} threshold: {threshold:.3f}")

plotROCs(fpr, tpr, roc_auc, users)
```

```
User 7 is being analyzed:
CV Accuracy: 0.90 (+/- 0.04)
Test Accuracy: 0.89
7: 1.000 threshold: 0.466
User 9 is being analyzed:
CV Accuracy: 0.95 (+/- 0.02)
Test Accuracy: 0.83
9: 1.000 threshold: 0.360
User 15 is being analyzed:
CV Accuracy: 0.76 (+/- 0.08)
Test Accuracy: 0.73
15: 0.984 threshold: 0.512
User 16 is being analyzed:
CV Accuracy: 0.72 (+/- 0.06)
Test Accuracy: 0.68
16: 0.973 threshold: 0.474
User 20 is being analyzed:
CV Accuracy: 0.84 (+/- 0.04)
Test Accuracy: 0.81
20: 0.985 threshold: 0.502
User 21 is being analyzed:
CV Accuracy: 0.76 (+/- 0.05)
Test Accuracy: 0.75
21: 0.975 threshold: 0.525
User 23 is being analyzed:
CV Accuracy: 0.77 (+/- 0.06)
Test Accuracy: 0.69
23: 0.952 threshold: 0.470
User 29 is being analyzed:
CV Accuracy: 0.81 (+/- 0.08)
Test Accuracy: 0.77
29: 0.987 threshold: 0.528
User 35 is being analyzed:
CV Accuracy: 0.74 (+/- 0.04)
Test Accuracy: 0.69
35: 0.970 threshold: 0.495
```



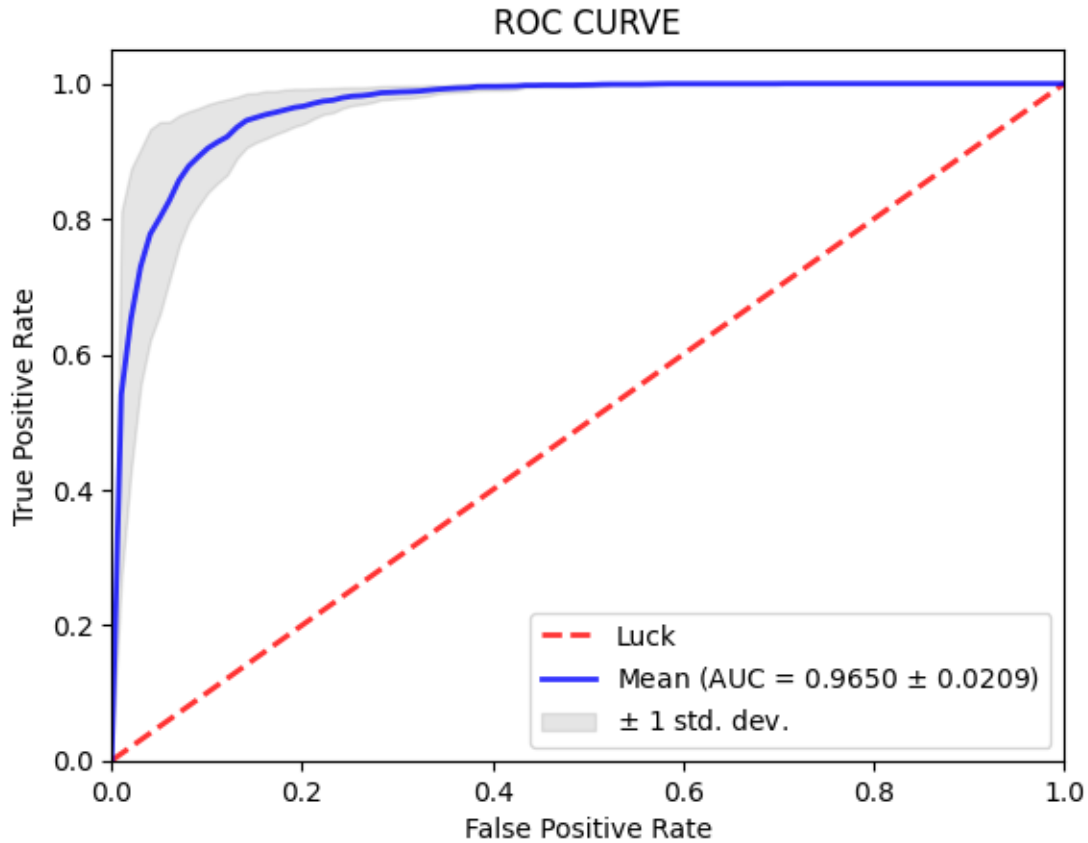
[ ]:
```
from SVMmodel import *
```

[ ]:
```
shuffle = True
fpr = {}
tpr = {}
roc_auc = {}

for legalUser in users:
    dataset = balabitAnalyser.createTrainingDataWithLabel(legalUser)
    X = dataset[:, 0:-1]
    y = dataset[:, -1]
    X_train, X_validation, y_train, y_validation = balabitAnalyser.
 ↪trainingTestSplit(X, y, TEST_SIZE, shuffle)
    model = SVMModel(dataset, users)
    fpr[legalUser], tpr[legalUser], thr = model.evaluate(X_train, y_train,␣
 ↪X_validation, y_validation, legalUser)
    threshold = -1
    try:
```

```
        eer = brentq(lambda x: 1. - x - interp1d(fpr[legalUser],␣
↪tpr[legalUser])(x), 0., 1.) ## brentq znajdowanie 0 , interpid interpolacja
        threshold = interp1d(fpr[legalUser], thr)(eer)

    except (ZeroDivisionError, ValueError):
        print("Division by zero")

    roc_auc[legalUser] = auc(fpr[legalUser], tpr[legalUser])
    print(f"{legalUser}: {roc_auc[legalUser]:.3f} threshold: {threshold:.3f}")


plotROCs(fpr, tpr, roc_auc, users)
```

```
User 7 is being analyzed:
CV Accuracy: 0.83 (+/- 0.04)
Test Accuracy: 0.83
7: 0.995 threshold: 0.521
User 9 is being analyzed:
CV Accuracy: 0.88 (+/- 0.04)
Test Accuracy: 0.89
9: 1.000 threshold: 0.558
User 15 is being analyzed:
CV Accuracy: 0.72 (+/- 0.06)
Test Accuracy: 0.72
15: 0.950 threshold: 0.516
User 16 is being analyzed:
CV Accuracy: 0.69 (+/- 0.09)
Test Accuracy: 0.73
16: 0.947 threshold: 0.541
User 20 is being analyzed:
CV Accuracy: 0.79 (+/- 0.04)
Test Accuracy: 0.78
20: 0.971 threshold: 0.501
User 21 is being analyzed:
CV Accuracy: 0.74 (+/- 0.05)
Test Accuracy: 0.72
21: 0.963 threshold: 0.534
User 23 is being analyzed:
CV Accuracy: 0.71 (+/- 0.04)
Test Accuracy: 0.75
23: 0.980 threshold: 0.505
User 29 is being analyzed:
CV Accuracy: 0.72 (+/- 0.08)
Test Accuracy: 0.73
29: 0.962 threshold: 0.484
User 35 is being analyzed:
CV Accuracy: 0.70 (+/- 0.08)
Test Accuracy: 0.71
```

```
35: 0.932 threshold: 0.507
```



As its visible the results for both of the algorithms are satisfactionary

2.2 Unsupervised algorithms

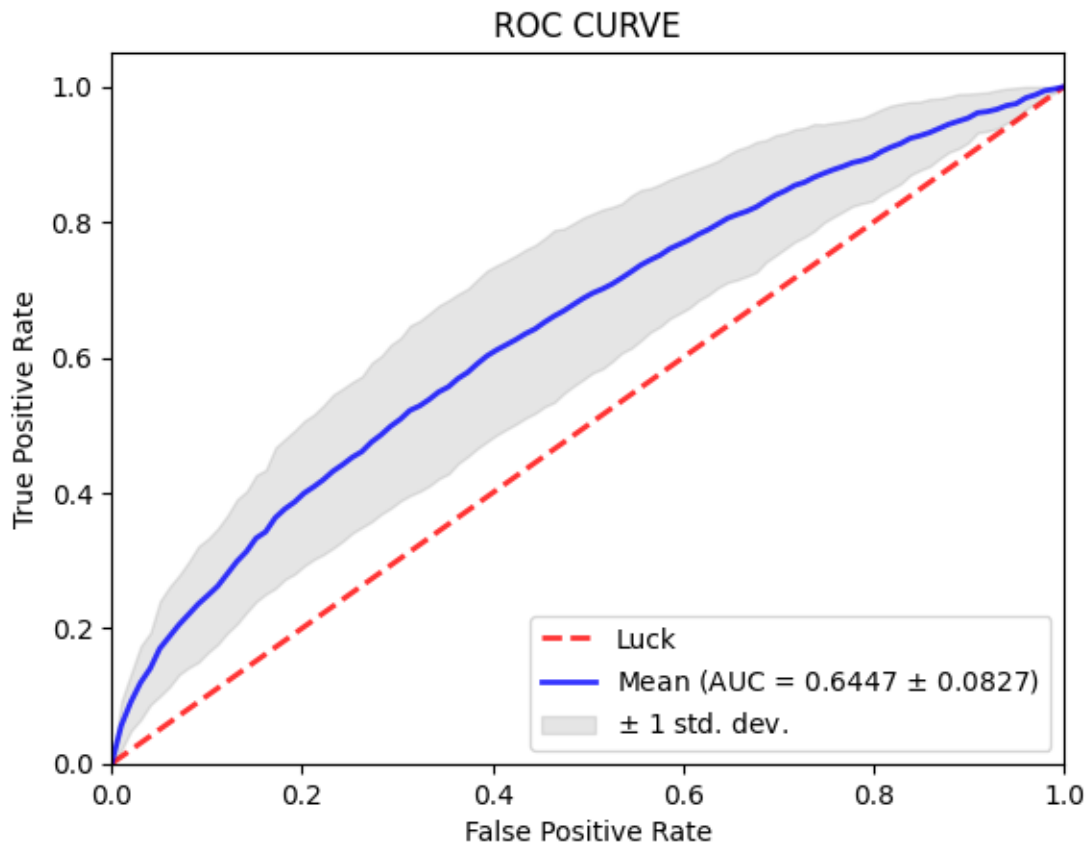
One class SVM and Isolation Forest

[ ]:
```
from OneClassSVMModel import *
```

[ ]:
```
fpr = {}
tpr = {}
roc_auc = {}
shuffle = True
for legalUser in users:
    dataset = balabitAnalyser.createTrainingDataWithLabel(legalUser)
    X = dataset[:, 0:-1]
    y = dataset[:, -1]
    X_train, X_validation, y_train, y_validation = balabitAnalyser.
 ↪trainingTestSplit(X, y, TEST_SIZE, shuffle)
    y_validation = np.where(y_validation == 0, -1, y_validation)
```

```
    model = OneClassSVMModel(dataset, users)
    fpr[legalUser], tpr[legalUser], thr = model.evaluate(X_train, y_train,␣
↪X_validation, y_validation, legalUser)
    threshold = -1
    try:
        eer = brentq(lambda x: 1. - x - interp1d(fpr[legalUser],␣
↪tpr[legalUser])(x), 0., 1.) ## brentq znajdowanie 0 , interpid interpolacja
        threshold = interp1d(fpr[legalUser], thr)(eer)

    except (ZeroDivisionError, ValueError):
        print("Division by zero")

    roc_auc[legalUser] = auc(fpr[legalUser], tpr[legalUser])
    print(str(legalUser) + ": " + str(roc_auc[legalUser])+" threshold:␣
↪"+str(threshold))

plotROCs(fpr, tpr, roc_auc, users)
```

```
User 7 is being analyzed:
Test Accuracy: 0.57
7: 0.7021912169177246 threshold: -28.28555165596241
User 9 is being analyzed:
Test Accuracy: 0.55
9: 0.694016498981914 threshold: -33.6279915465279
User 15 is being analyzed:
Test Accuracy: 0.54
15: 0.5885021008987952 threshold: -16.135873572702405
User 16 is being analyzed:
Test Accuracy: 0.57
16: 0.6391230666924084 threshold: -17.791534814384487
User 20 is being analyzed:
Test Accuracy: 0.51
20: 0.5365326243844456 threshold: -47.00445140185061
User 21 is being analyzed:
Test Accuracy: 0.57
21: 0.6899810370593629 threshold: -38.542291712742745
User 23 is being analyzed:
Test Accuracy: 0.59
23: 0.7247941353685479 threshold: -41.39693256675982
User 29 is being analyzed:
Test Accuracy: 0.52
29: 0.4896003697646306 threshold: -48.26315867527007
User 35 is being analyzed:
Test Accuracy: 0.58
35: 0.736000758397175 threshold: -38.04377268917362
```

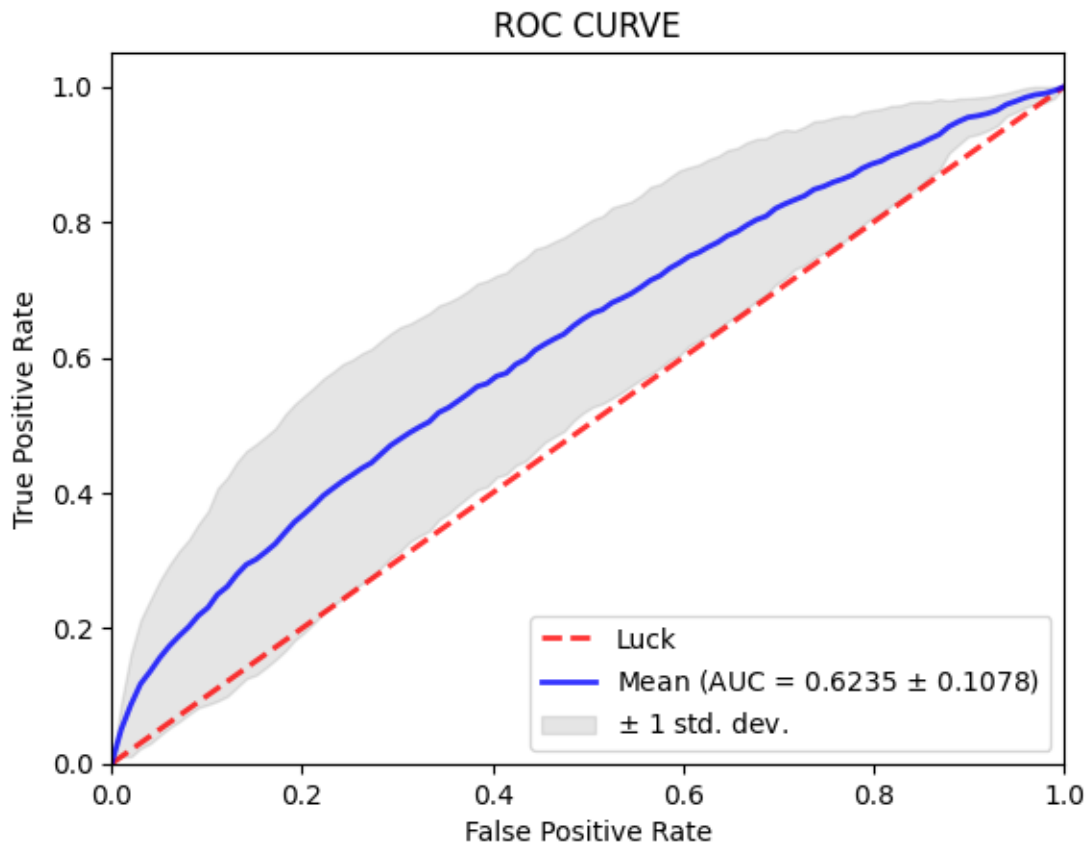


[ ]:
```
from IsolationForestModel import *
```

[ ]:
```
fpr = {}
tpr = {}
roc_auc = {}
shuffle = True
for legalUser in users:
    dataset = balabitAnalyser.createTrainingDataWithLabel(legalUser)
    X = dataset[:, 0:-1]
    y = dataset[:, -1]
    X_train, X_validation, y_train, y_validation = balabitAnalyser.
 ↪trainingTestSplit(X, y, TEST_SIZE, shuffle)
    y_validation = np.where(y_validation == 0, -1, y_validation)

    model = IsolationForestModel(dataset, users)
    fpr[legalUser], tpr[legalUser], thr = model.evaluate(X_train, y_train,␣
 ↪X_validation, y_validation, legalUser)
    threshold = -1
    try:
```

```
        eer = brentq(lambda x: 1. - x - interp1d(fpr[legalUser],␣
↪tpr[legalUser])(x), 0., 1.) ## brentq znajdowanie 0 , interpid interpolacja
        threshold = interp1d(fpr[legalUser], thr)(eer)

    except (ZeroDivisionError, ValueError):
        print("Division by zero")

    roc_auc[legalUser] = auc(fpr[legalUser], tpr[legalUser])
    print(str(legalUser) + ": " + str(roc_auc[legalUser])+" threshold:␣
↪"+str(threshold))

plotROCs(fpr, tpr, roc_auc, users)
```

```
User 7 is being analyzed:
Test Accuracy: 0.52
7: 0.48689904934359435 threshold: 0.06588043219408114
User 9 is being analyzed:
Test Accuracy: 0.55
9: 0.6680584238723773 threshold: 0.0745177234732044
User 15 is being analyzed:
Test Accuracy: 0.55
15: 0.6280030551312318 threshold: 0.044518790700259055
User 16 is being analyzed:
Test Accuracy: 0.52
16: 0.6276537171704171 threshold: 0.08209785528790253
User 20 is being analyzed:
Test Accuracy: 0.50
20: 0.4055601111385331 threshold: 0.0807837079534502
User 21 is being analyzed:
Test Accuracy: 0.55
21: 0.733670451412387 threshold: 0.06411005959648751
User 23 is being analyzed:
Test Accuracy: 0.56
23: 0.7586295982661864 threshold: 0.061191774745724764
User 29 is being analyzed:
Test Accuracy: 0.49
29: 0.6033728252748426 threshold: 0.10147664237500807
User 35 is being analyzed:
Test Accuracy: 0.56
35: 0.6986055599964474 threshold: 0.05205284941895097
```

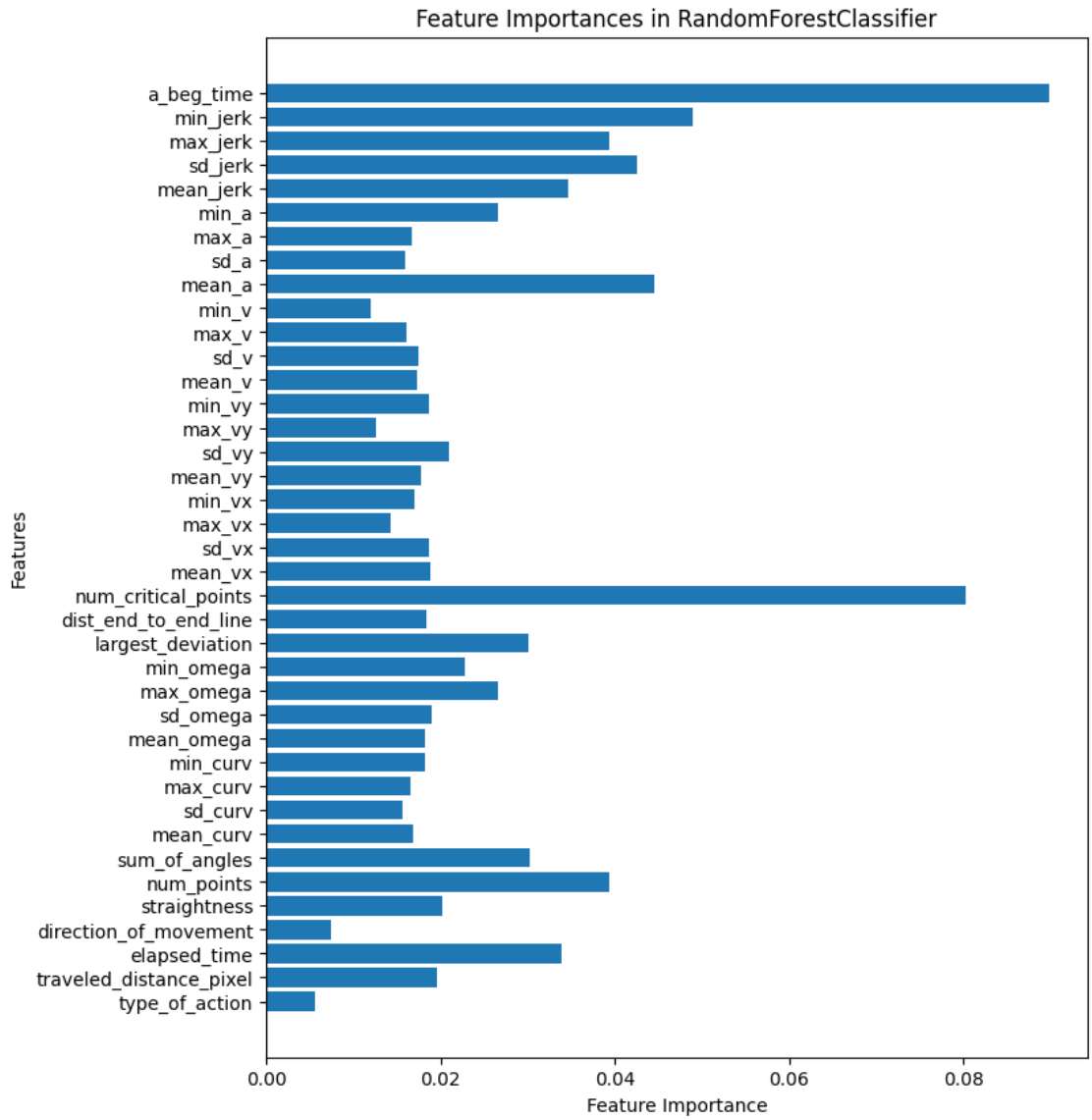


3 Checking which features are important for Random Forest

4 Feature importance method

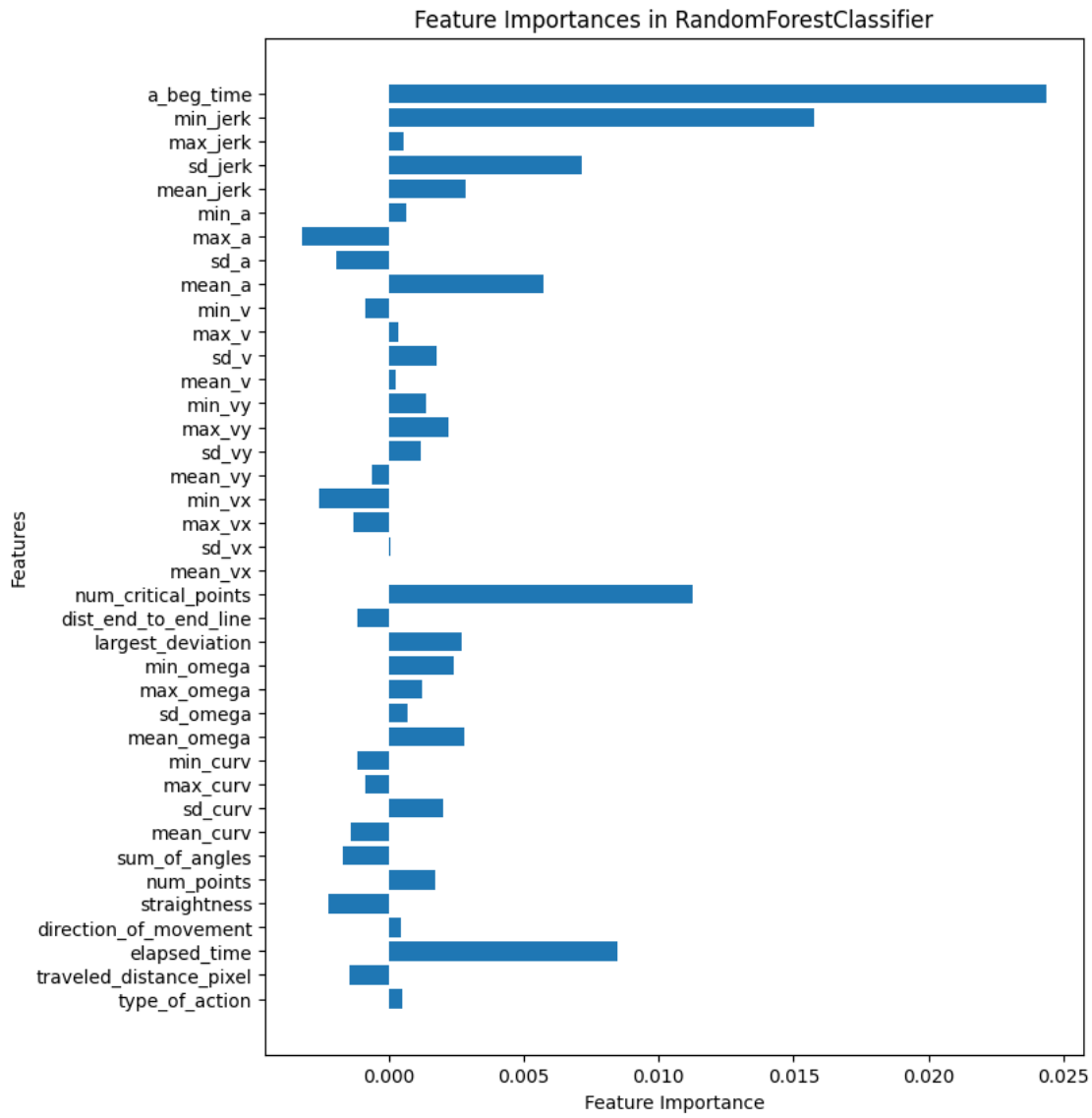
[ ]:
```
## choose legalUser randomly and create dataset for him ##
legalUser = random.choice(users)
dataset = balabitAnalyser.createTrainingDataWithLabel(legalUser)
X = dataset[:, 0:-1]
y = dataset[:, -1]
X_train, X_validation, y_train, y_validation = balabitAnalyser.
 ↪trainingTestSplit(X, y, TEST_SIZE, shuffle)
model = RandomForestModel(dataset, users)
feature_importance = model.calculateFeatureImportance(X_train, y_train,␣
 ↪X_validation)
df1 = balabitAnalyser.returnDataframe()
```

[ ]:
```
model.plotFeatureImportance(feature_importance, X, df1)
```



4.1 Permutation Importance method

```
[ ]: permutationImportance = model.calculatePermutationImportance(X_train, y_train,␣
     ↪X_validation, y_validation)
     model.plotFeatureImportance(permutationImportance, X, df1)
```



The most importants variables are : - a_beg_time (acceleration in the begginng), - jerk variables, - number_of_critical_points (the amount of times the mouse is changing the direction) - acceleration, - straightness.

It all depends on the user, yet summarazing those are the ones which contribute the most.

5

6 Creating datasets using the test files too (works only for BALABIT)

Labeling is being done during feature extraction and not in the end of the process

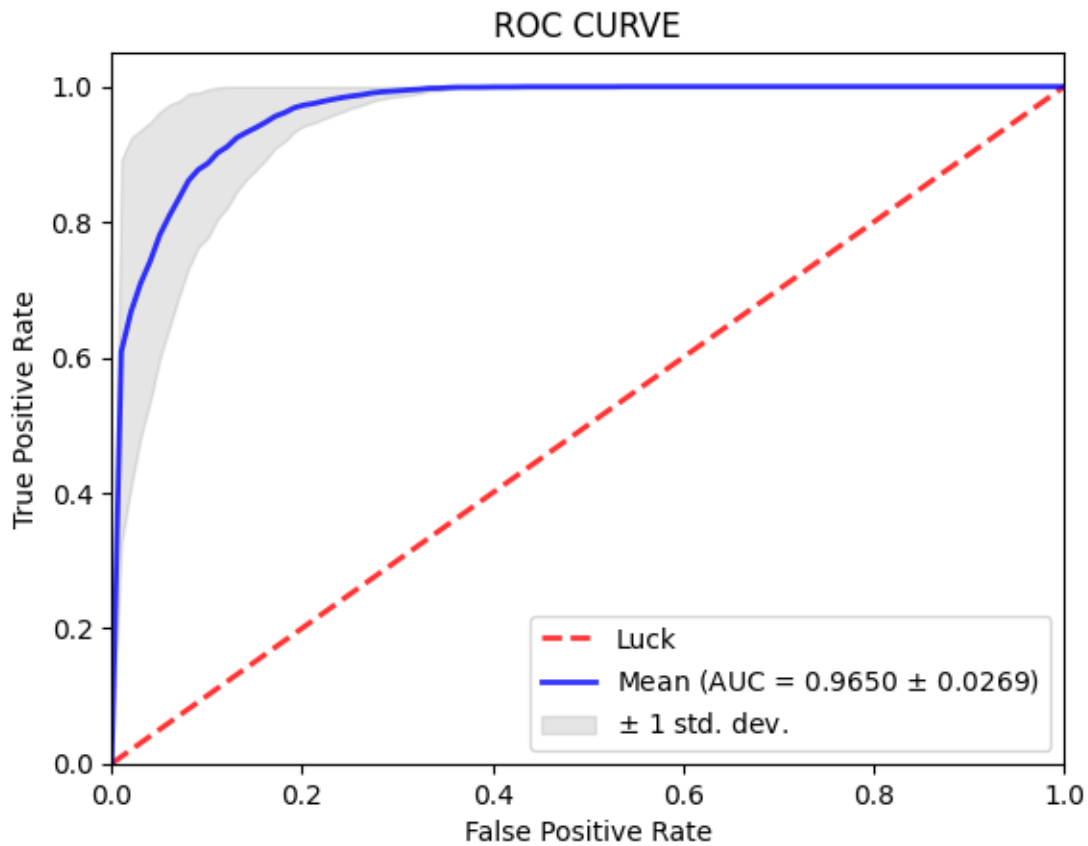
6.1 Random Forest

[ ]:
```
users = [7,9,15,16,20,21,23,29, 35]
balabit_reader = DataReaderBalabit(BALABIT, users, True, limit = 1000) ## the␣
↪arguments are DATASET, users, supervised = True, how many records should be␣
↪procesed ##
```

[ ]:
```
shuffle = True
## dictionaries of fpr tpr ##
fpr = {}
tpr = {}
roc_auc = {}

for legalUser in users:
    # the function below has 3 variables - legalser (which of the sessions are␣
↪legal), training - use training files only, test - use test files and parse␣
↪CSV to check the legality ##
    balabit_reader.processDataWithLabels(legalUser, training = True, test =␣
↪True)
    # the rest is the same as in previous attempts ##
    path = balabit_reader.getFileName()
    legality_analyser = legalityAnalyser(path)
    ## balanced = True - it will create balanced dataset // when balanced is␣
↪False then  every false action of the dataset will be used, ratio legal to␣
↪illegal will be  something like 1:8
    dataset = legality_analyser.createTrainingData(balanced = True)

    ## the last label is 0 and 1
    X = dataset[:, 0:-1]
    y = dataset[:, -1]

    X_train, X_validation, y_train, y_validation = legality_analyser.
↪trainingTestSplit(X, y, TEST_SIZE, shuffle)
    model = RandomForestModel(dataset, users)
    ## evaluating the model, the variables are X_train, y_train, X_validation,␣
↪y_validation, scale --> use standardScler, and legalUser
    fpr[legalUser], tpr[legalUser], thr = model.evaluate(X_train, y_train,␣
↪X_validation, y_validation, scale = True, user = legalUser)
    threshold = -1
    try:
        eer = brentq(lambda x: 1. - x - interp1d(fpr[legalUser],␣
↪tpr[legalUser])(x), 0., 1.) ## brentq znajdowanie 0 , interpid interpolacja
        threshold = interp1d(fpr[legalUser], thr)(eer)

    except (ZeroDivisionError, ValueError):
        print("Division by zero")
```

```
    roc_auc[legalUser] = auc(fpr[legalUser], tpr[legalUser])
    print(f"{legalUser}: {roc_auc[legalUser]:.3f} threshold: {threshold:.3f}")

plotROCs(fpr, tpr, roc_auc, users)
```

```
User 7 is being analyzed:
CV Accuracy: 0.93 (+/- 0.03)
Test Accuracy: 0.75
7: 0.999 threshold: 0.346
User 9 is being analyzed:
CV Accuracy: 0.96 (+/- 0.03)
Test Accuracy: 0.94
9: 1.000 threshold: 0.632
User 15 is being analyzed:
CV Accuracy: 0.74 (+/- 0.06)
Test Accuracy: 0.70
15: 0.965 threshold: 0.495
User 16 is being analyzed:
CV Accuracy: 0.69 (+/- 0.04)
Test Accuracy: 0.68
16: 0.952 threshold: 0.501
User 20 is being analyzed:
CV Accuracy: 0.83 (+/- 0.04)
Test Accuracy: 0.81
20: 0.989 threshold: 0.507
User 21 is being analyzed:
CV Accuracy: 0.72 (+/- 0.04)
Test Accuracy: 0.72
21: 0.959 threshold: 0.496
User 23 is being analyzed:
CV Accuracy: 0.71 (+/- 0.05)
Test Accuracy: 0.67
23: 0.921 threshold: 0.496
User 29 is being analyzed:
CV Accuracy: 0.76 (+/- 0.05)
Test Accuracy: 0.76
29: 0.987 threshold: 0.490
User 35 is being analyzed:
CV Accuracy: 0.67 (+/- 0.06)
Test Accuracy: 0.64
35: 0.934 threshold: 0.458
```



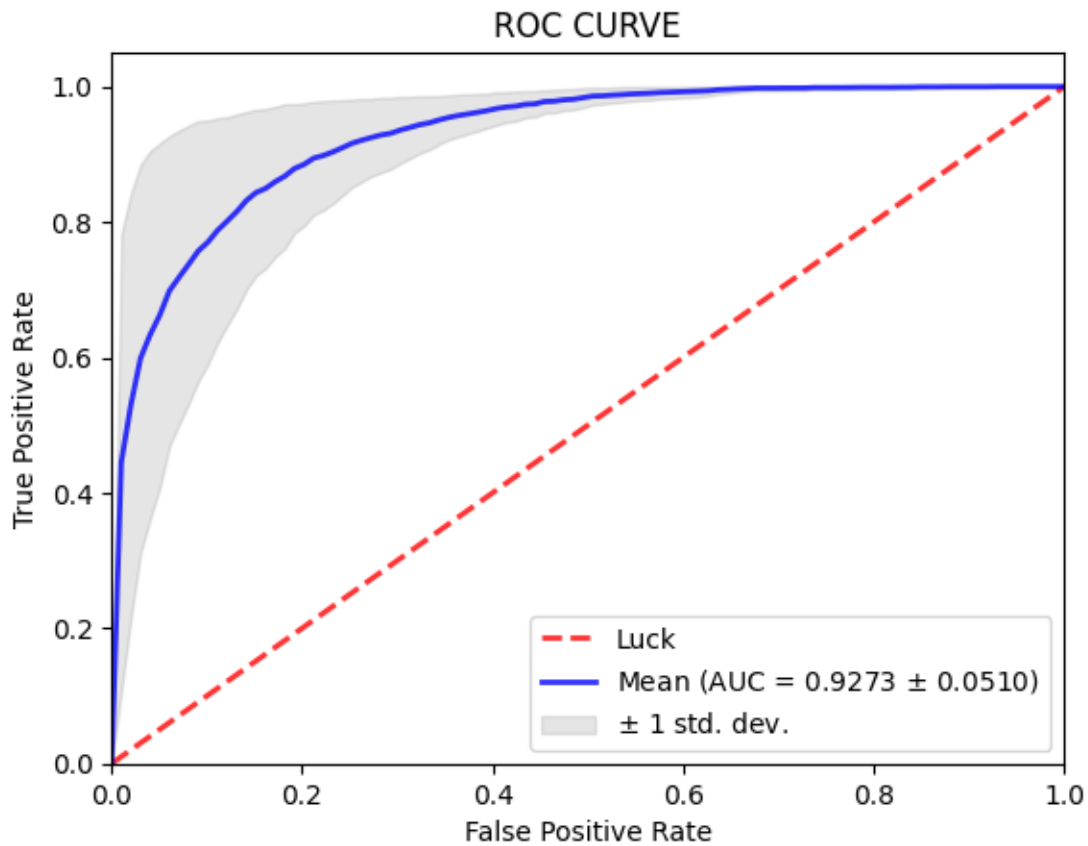
6.2 SVM

[ ]:
```
shuffle = True
## dictionaries of fpr tpr ##
fpr = {}
tpr = {}
roc_auc = {}

for legalUser in users:
    # the function below has 3 variables - legalser (which of the sessions are␣
↪legal), training - use training files only, test - use test files and parse␣
↪CSV to check the legality ##
    balabit_reader.processDataWithLabels(legalUser, training = True, test =␣
↪True)
    # the rest is the same as in previous attempts ##
    path = balabit_reader.getFileName()
    legality_analyser = legalityAnalyser(path)
```

```
    ## balanced = True - it will create balanced dataset // when balanced is␣
↪False then  every false action of the dataset will be used, ratio legal to␣
↪illegal will be  something like 1:8
    dataset = legality_analyser.createTrainingData(balanced = True)

    ## the last label is 0 and 1
    X = dataset[:, 0:-1]
    y = dataset[:, -1]

    X_train, X_validation, y_train, y_validation = legality_analyser.
↪trainingTestSplit(X, y, TEST_SIZE, shuffle)
    model = SVMModel(dataset, users)
    ## evaluating the model, the variables are X_train, y_train, X_validation,␣
↪y_validation, scale --> use standardScler, and legalUser
    fpr[legalUser], tpr[legalUser], thr = model.evaluate(X_train, y_train,␣
↪X_validation, y_validation, user = legalUser)
    threshold = -1
    try:
        eer = brentq(lambda x: 1. - x - interp1d(fpr[legalUser],␣
↪tpr[legalUser])(x), 0., 1.) ## brentq znajdowanie 0 , interpid interpolacja
        threshold = interp1d(fpr[legalUser], thr)(eer)

    except (ZeroDivisionError, ValueError):
        print("Division by zero")

    roc_auc[legalUser] = auc(fpr[legalUser], tpr[legalUser])
    print(f"{legalUser}: {roc_auc[legalUser]:.3f} threshold: {threshold:.3f}")

plotROCs(fpr, tpr, roc_auc, users)
```

```
File already exist
User 7 is being analyzed:
CV Accuracy: 0.88 (+/- 0.03)
Test Accuracy: 0.89
7: 0.999 threshold: 0.559
File already exist
User 9 is being analyzed:
CV Accuracy: 0.91 (+/- 0.04)
Test Accuracy: 0.92
9: 1.000 threshold: 0.512
File already exist
User 15 is being analyzed:
CV Accuracy: 0.69 (+/- 0.04)
Test Accuracy: 0.67
15: 0.915 threshold: 0.518
File already exist
User 16 is being analyzed:
```

```
CV Accuracy: 0.64 (+/- 0.04)
Test Accuracy: 0.66
16: 0.874 threshold: 0.536
File already exist
User 20 is being analyzed:
CV Accuracy: 0.79 (+/- 0.04)
Test Accuracy: 0.78
20: 0.969 threshold: 0.493
File already exist
User 21 is being analyzed:
CV Accuracy: 0.66 (+/- 0.05)
Test Accuracy: 0.66
21: 0.904 threshold: 0.518
File already exist
User 23 is being analyzed:
CV Accuracy: 0.66 (+/- 0.04)
Test Accuracy: 0.69
23: 0.894 threshold: 0.497
File already exist
User 29 is being analyzed:
CV Accuracy: 0.72 (+/- 0.04)
Test Accuracy: 0.73
29: 0.955 threshold: 0.515
File already exist
User 35 is being analyzed:
CV Accuracy: 0.68 (+/- 0.04)
Test Accuracy: 0.65
35: 0.850 threshold: 0.530
```



7 CHAOSHEN

[ ]:
```
from utils.consts import *
from utils. plotting import *
from data_reader_chaoshen import *
from nonlegality_analyser import *
import random
```

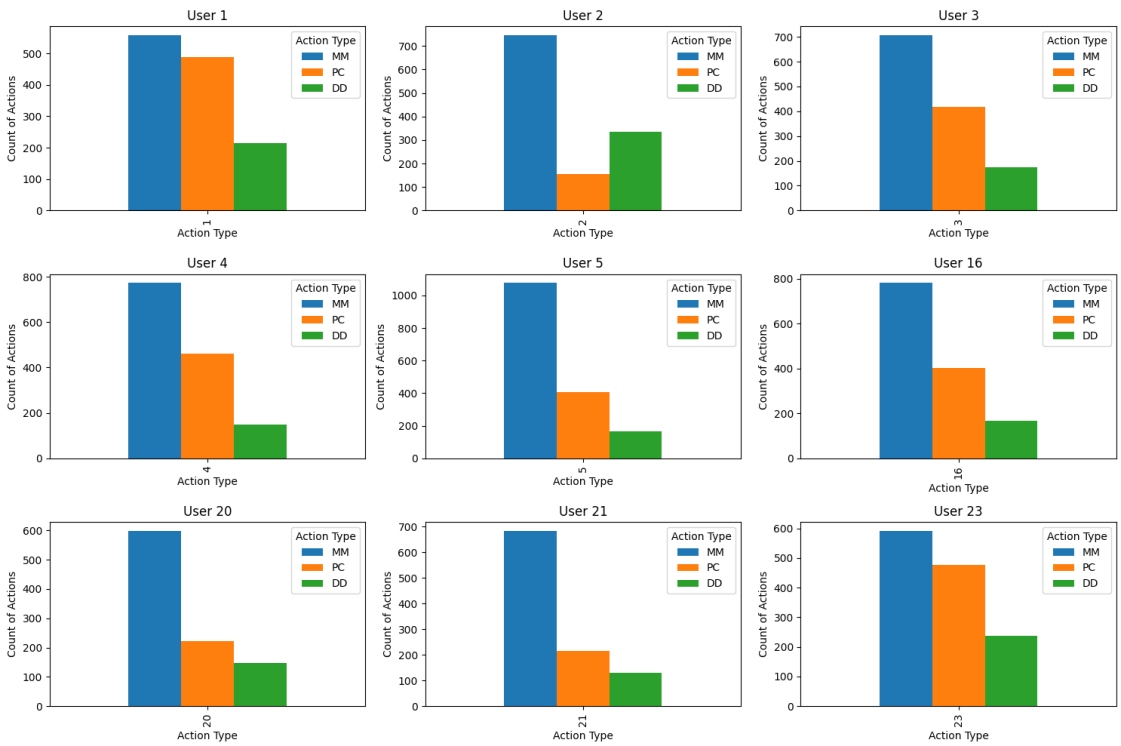
[ ]:
```
users = [1,2, 3, 5, 4,16,20,21,23] # a few users
# users = [i for i in range (1,29)] ## all users
chaoshen_reader = DataReaderChaoShen(CHAOSHEN, users, limit = 1000)
chaoshen_reader.processDataWithoutLabels()
```

[ ]:
```
path = chaoshen_reader.getFileName()
chaoShenAnalyser = nonLegalityAnalyser(path)
print(chaoShenAnalyser.countActions())
```

```
type_of_action     1    3    4
userid
1                558  488  214
```

```
2                 746   154   334
3                 707   419   173
4                 773   462   148
5                1077   407   167
16                781   402   166
20                599   221   148
21                684   214   130
23                592   477   238
```

[ ]:
```
chaoShenAnalyser.plotActionHistograms()
```



7.1 Random Forest

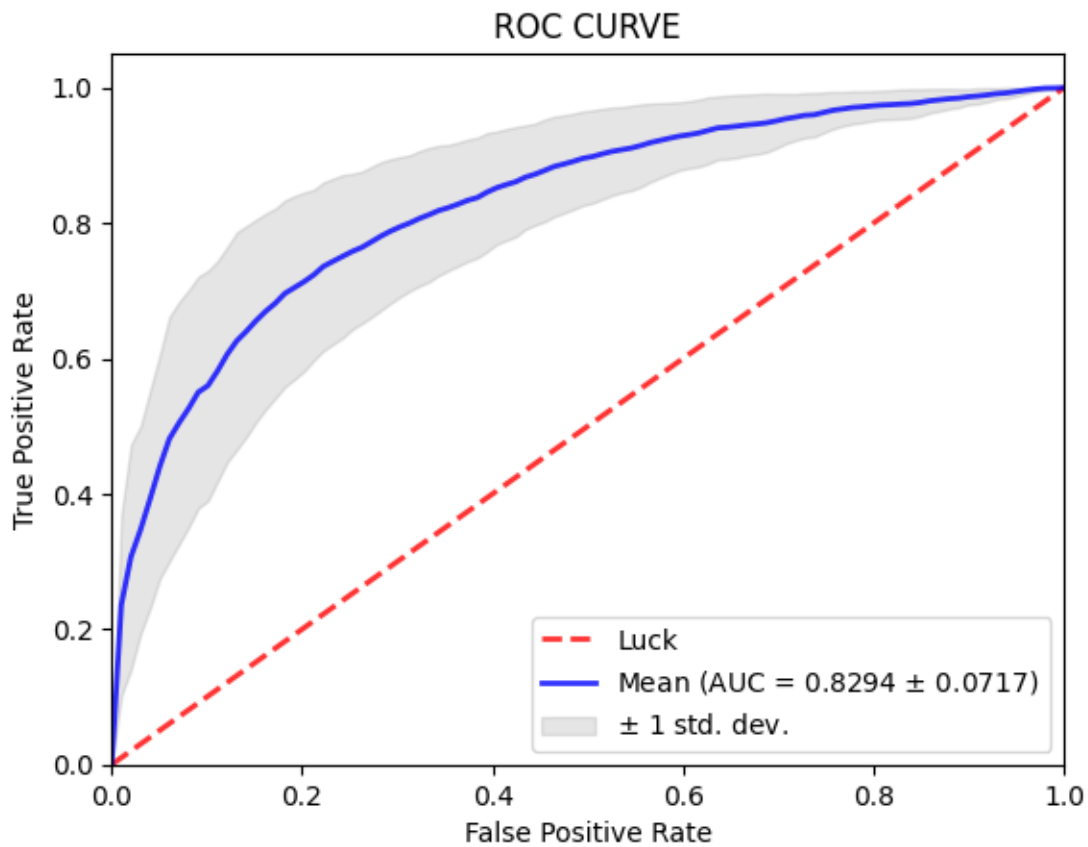
[ ]:
```
from RandomForestModel import *
```

[ ]:
```
shuffle = True
fpr = {}
tpr = {}
roc_auc = {}
for legalUser in users:
    dataset = chaoShenAnalyser.createTrainingDataWithLabel(legalUser)
    X = dataset[:, 0:-1]
    y = dataset[:, -1]
```

```
    X_train, X_validation, y_train, y_validation = chaoShenAnalyser.
↪trainingTestSplit(X, y, TEST_SIZE, shuffle)
    model = RandomForestModel(dataset, users)
    fpr[legalUser], tpr[legalUser], thr = model.evaluate(X_train, y_train,␣
↪X_validation, y_validation, scale = True, user = legalUser)
    threshold = -1
    try:
        eer = brentq(lambda x: 1. - x - interp1d(fpr[legalUser],␣
↪tpr[legalUser])(x), 0., 1.) ## brentq znajdowanie 0 , interpid interpolacja
        threshold = interp1d(fpr[legalUser], thr)(eer)

    except (ZeroDivisionError, ValueError):
        print("Division by zero")

    roc_auc[legalUser] = auc(fpr[legalUser], tpr[legalUser])
    print(f"{legalUser}: {roc_auc[legalUser]:.3f} threshold: {threshold:.3f}")

plotROCs(fpr, tpr, roc_auc, users)
```

```
User 1 is being analyzed:
CV Accuracy: 0.78 (+/- 0.06)
Test Accuracy: 0.57
1: 0.819 threshold: 0.430
User 2 is being analyzed:
CV Accuracy: 0.80 (+/- 0.06)
Test Accuracy: 0.64
2: 0.829 threshold: 0.460
User 3 is being analyzed:
CV Accuracy: 0.72 (+/- 0.06)
Test Accuracy: 0.56
3: 0.690 threshold: 0.418
User 5 is being analyzed:
CV Accuracy: 0.75 (+/- 0.04)
Test Accuracy: 0.52
5: 0.819 threshold: 0.402
User 4 is being analyzed:
CV Accuracy: 0.72 (+/- 0.07)
Test Accuracy: 0.62
4: 0.820 threshold: 0.515
User 16 is being analyzed:
CV Accuracy: 0.73 (+/- 0.06)
Test Accuracy: 0.54
16: 0.862 threshold: 0.588
User 20 is being analyzed:
CV Accuracy: 0.72 (+/- 0.11)
Test Accuracy: 0.59
20: 0.779 threshold: 0.503
```

```
User 21 is being analyzed:
CV Accuracy: 0.81 (+/- 0.06)
Test Accuracy: 0.60
21: 0.971 threshold: 0.425
User 23 is being analyzed:
CV Accuracy: 0.77 (+/- 0.05)
Test Accuracy: 0.64
23: 0.882 threshold: 0.526
```



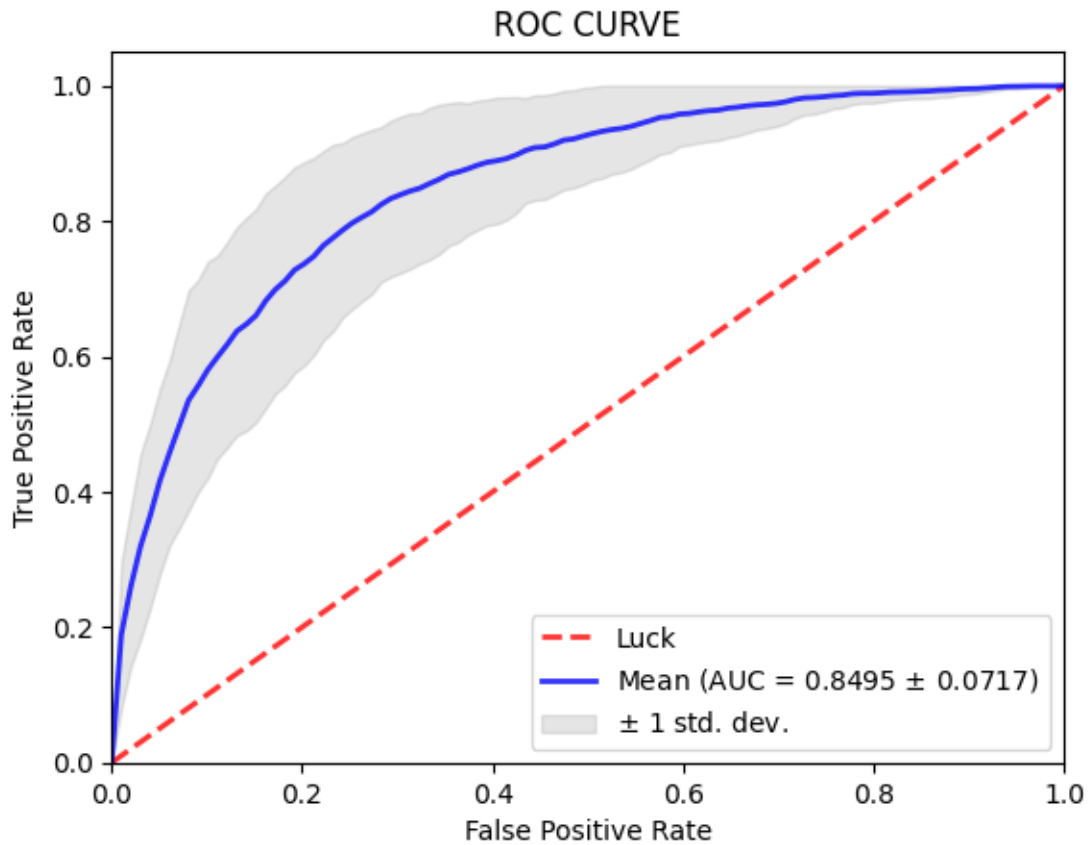
7.2 SVM

[ ]:
```
shuffle = True
fpr = {}
tpr = {}
roc_auc = {}

for legalUser in users:
    dataset = chaoShenAnalyser.createTrainingDataWithLabel(legalUser)
    X = dataset[:, 0:-1]
    y = dataset[:, -1]
```

```
    X_train, X_validation, y_train, y_validation = chaoShenAnalyser.
↪trainingTestSplit(X, y, TEST_SIZE, shuffle)
    model = SVMModel(dataset, users)
    fpr[legalUser], tpr[legalUser], thr = model.evaluate(X_train, y_train,␣
↪X_validation, y_validation, legalUser)
    threshold = -1
    try:
        eer = brentq(lambda x: 1. - x - interp1d(fpr[legalUser],␣
↪tpr[legalUser])(x), 0., 1.) ## brentq znajdowanie 0 , interpid interpolacja
        threshold = interp1d(fpr[legalUser], thr)(eer)

    except (ZeroDivisionError, ValueError):
        print("Division by zero")

    roc_auc[legalUser] = auc(fpr[legalUser], tpr[legalUser])
    print(f"{legalUser}: {roc_auc[legalUser]:.3f} threshold: {threshold:.3f}")


plotROCs(fpr, tpr, roc_auc, users)
```

```
User 1 is being analyzed:
CV Accuracy: 0.60 (+/- 0.04)
Test Accuracy: 0.67
1: 0.901 threshold: 0.504
User 2 is being analyzed:
CV Accuracy: 0.69 (+/- 0.06)
Test Accuracy: 0.70
2: 0.937 threshold: 0.502
User 3 is being analyzed:
CV Accuracy: 0.59 (+/- 0.04)
Test Accuracy: 0.58
3: 0.678 threshold: 0.500
User 5 is being analyzed:
CV Accuracy: 0.65 (+/- 0.04)
Test Accuracy: 0.67
5: 0.890 threshold: 0.509
User 4 is being analyzed:
CV Accuracy: 0.62 (+/- 0.07)
Test Accuracy: 0.67
4: 0.865 threshold: 0.507
User 16 is being analyzed:
CV Accuracy: 0.61 (+/- 0.06)
Test Accuracy: 0.63
16: 0.848 threshold: 0.496
User 20 is being analyzed:
CV Accuracy: 0.62 (+/- 0.09)
Test Accuracy: 0.62
```

```
20: 0.793 threshold: 0.509
User 21 is being analyzed:
CV Accuracy: 0.65 (+/- 0.06)
Test Accuracy: 0.68
21: 0.889 threshold: 0.514
User 23 is being analyzed:
CV Accuracy: 0.65 (+/- 0.10)
Test Accuracy: 0.63
23: 0.847 threshold: 0.498
```



8 SINGAPUR

[ ]:
```
from utils.consts import *
from utils. plotting import *
from data_reader_singapur import *
from nonlegality_analyser import *
```

[ ]:
```
users = [1,2, 3, 4, 5, 6,9, 7, 16,20,21,23]
# users = [i for i in range (0,24)]
```

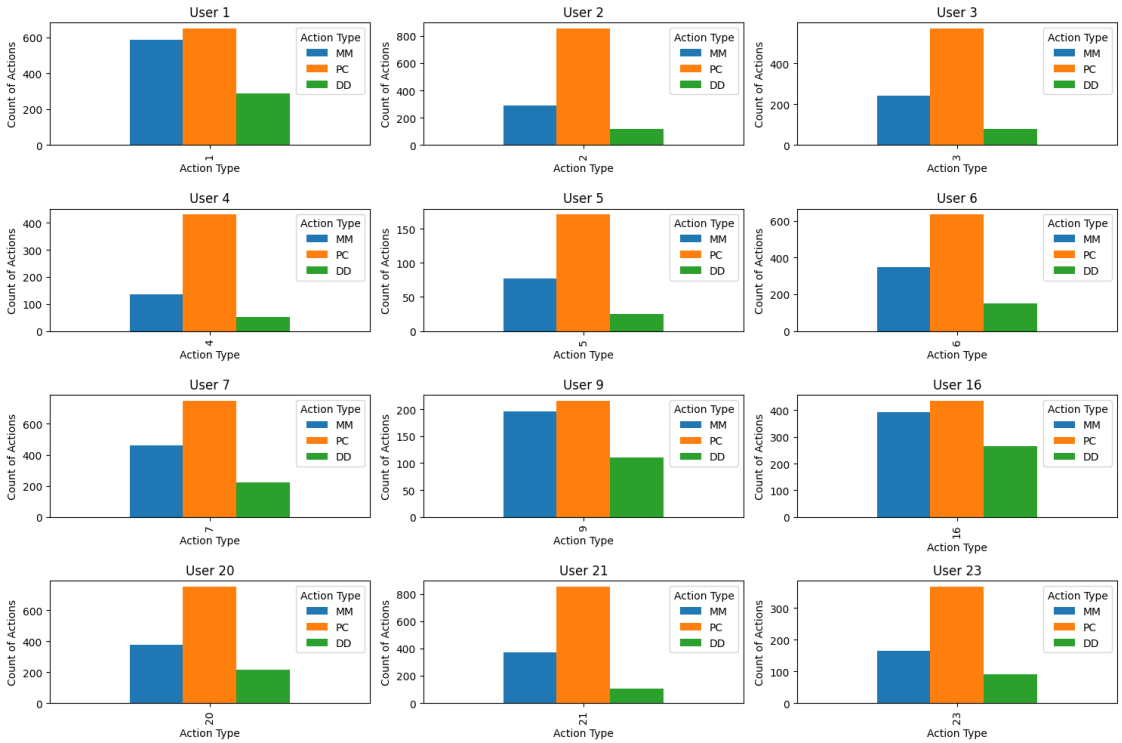
```
singapur_reader = DataReaderSingapur(SINGAPUR, users, limit = 1000)
```

[ ]:
```
singapur_reader.processDataWithoutLabels()
```

[ ]:
```
path = singapur_reader.getFileName()
singapurAnalyser = nonLegalityAnalyser(path)
print(singapurAnalyser.countActions())
```

```
type_of_action    1    3    4
userid
1               588  649  286
2               291  854  119
3               240  571   78
4               136  431   51
5                77  171   25
6               346  634  150
7               459  750  223
9               196  216  110
16              393  436  266
20              378  753  216
21              372  854  104
23              165  368   90
```

[ ]:
```
singapurAnalyser.plotActionHistograms()
```



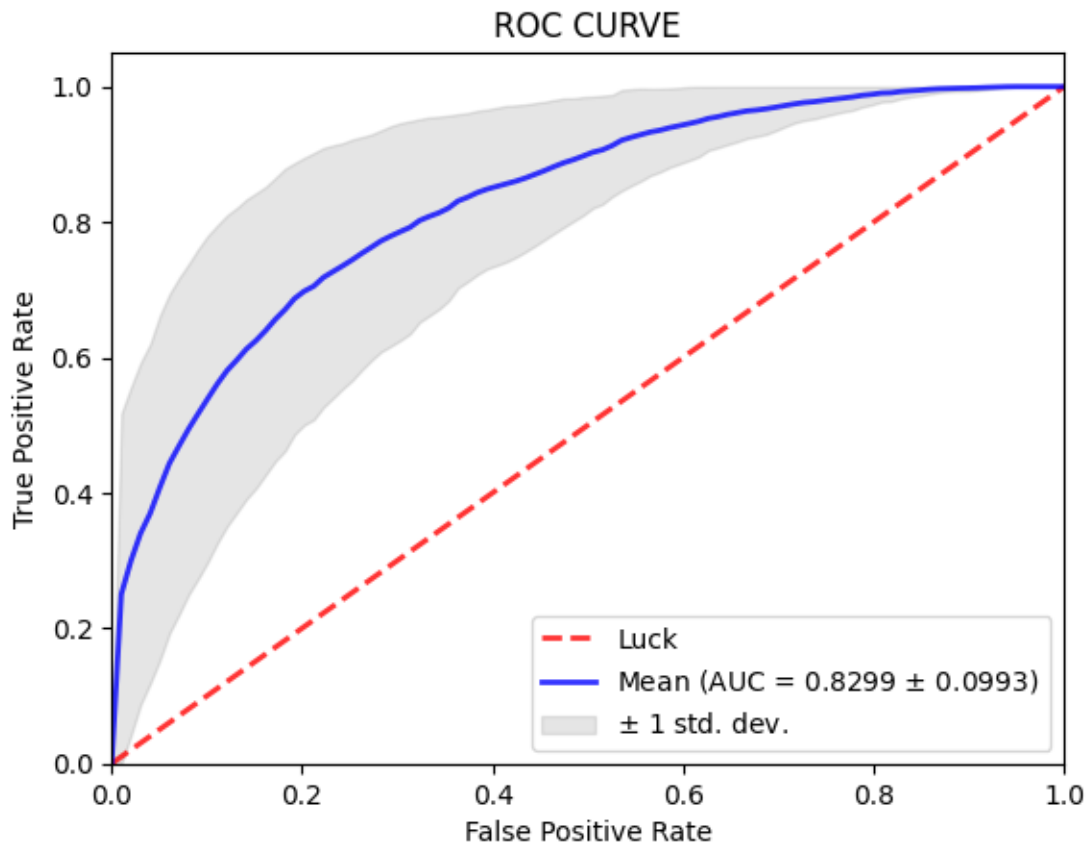
8.1 Random Forest

[ ]:
```
from RandomForestModel import *
```

[ ]:
```
shuffle = True
fpr = {}
tpr = {}
roc_auc = {}
for legalUser in users:
    dataset = singapurAnalyser.createTrainingDataWithLabel(legalUser)
    X = dataset[:, 0:-1]
    y = dataset[:, -1]
    X_train, X_validation, y_train, y_validation = singapurAnalyser.
 ↪trainingTestSplit(X, y, TEST_SIZE, shuffle)
    model = RandomForestModel(dataset, users)
    fpr[legalUser], tpr[legalUser], thr = model.evaluate(X_train, y_train,␣
 ↪X_validation, y_validation, scale = True, user = legalUser)
    threshold = -1
    try:
        eer = brentq(lambda x: 1. - x - interp1d(fpr[legalUser],␣
 ↪tpr[legalUser])(x), 0., 1.) ## brentq znajdowanie 0 , interpid interpolacja
        threshold = interp1d(fpr[legalUser], thr)(eer)

    except (ZeroDivisionError, ValueError):
        print("Division by zero")

    roc_auc[legalUser] = auc(fpr[legalUser], tpr[legalUser])
    print(f"{legalUser}: {roc_auc[legalUser]:.3f} threshold: {threshold:.3f}")

plotROCs(fpr, tpr, roc_auc, users)
```

```
User 1 is being analyzed:
CV Accuracy: 0.69 (+/- 0.05)
Test Accuracy: 0.54
1: 0.705 threshold: 0.355
User 2 is being analyzed:
CV Accuracy: 0.74 (+/- 0.06)
Test Accuracy: 0.50
2: 0.681 threshold: 0.356
User 3 is being analyzed:
CV Accuracy: 0.72 (+/- 0.07)
Test Accuracy: 0.61
3: 0.849 threshold: 0.430
User 4 is being analyzed:
CV Accuracy: 0.72 (+/- 0.10)
Test Accuracy: 0.59
4: 0.852 threshold: 0.414
User 5 is being analyzed:
```

```
CV Accuracy: 0.73 (+/- 0.11)
Test Accuracy: 0.76
5: 0.933 threshold: 0.455
User 6 is being analyzed:
CV Accuracy: 0.66 (+/- 0.05)
Test Accuracy: 0.51
6: 0.670 threshold: 0.296
User 9 is being analyzed:
CV Accuracy: 0.77 (+/- 0.07)
Test Accuracy: 0.63
9: 0.832 threshold: 0.446
User 7 is being analyzed:
CV Accuracy: 0.77 (+/- 0.05)
Test Accuracy: 0.62
7: 0.900 threshold: 0.423
User 16 is being analyzed:
CV Accuracy: 0.95 (+/- 0.02)
Test Accuracy: 0.59
16: 0.993 threshold: 0.635
User 20 is being analyzed:
CV Accuracy: 0.77 (+/- 0.05)
Test Accuracy: 0.59
20: 0.770 threshold: 0.577
User 21 is being analyzed:
CV Accuracy: 0.80 (+/- 0.04)
Test Accuracy: 0.70
21: 0.906 threshold: 0.500
User 23 is being analyzed:
CV Accuracy: 0.69 (+/- 0.12)
Test Accuracy: 0.59
23: 0.880 threshold: 0.418
```



8.2 SVM

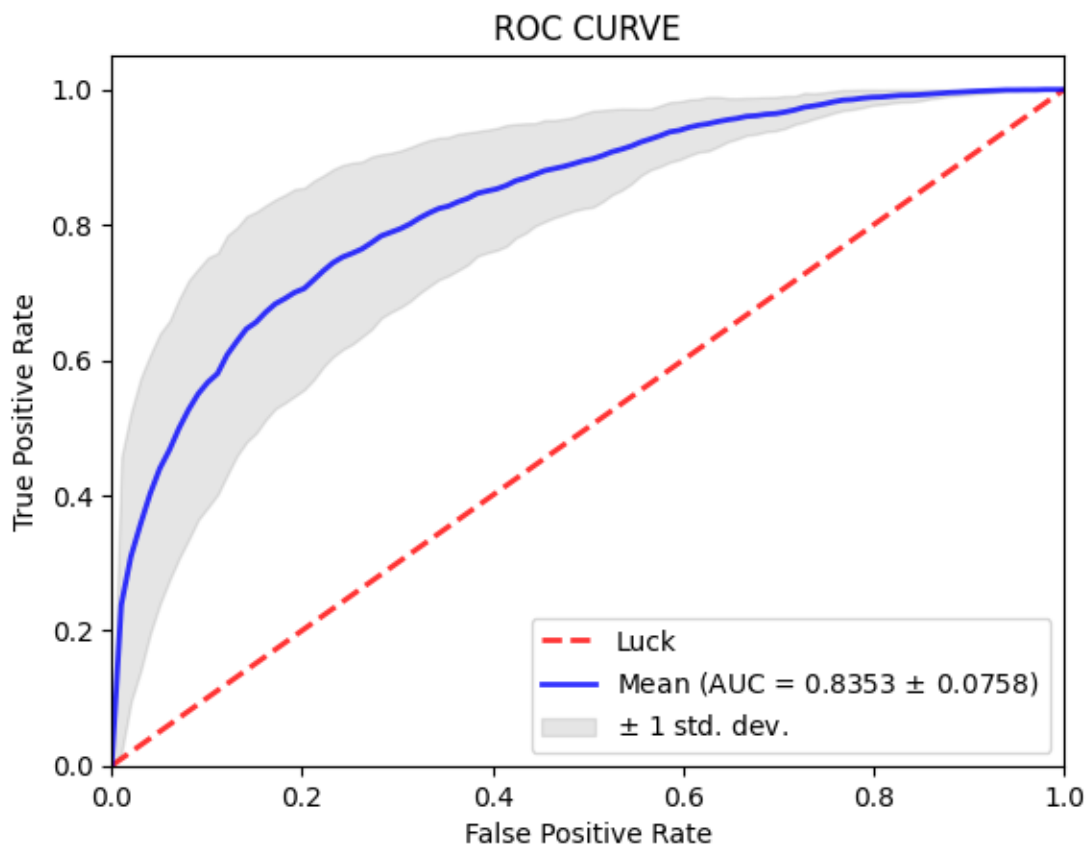
[ ]:
```
from SVMmodel import *
```

[ ]:
```
shuffle = True
fpr = {}
tpr = {}
roc_auc = {}
for legalUser in users:
    dataset = singapurAnalyser.createTrainingDataWithLabel(legalUser)
    X = dataset[:, 0:-1]
    y = dataset[:, -1]
    X_train, X_validation, y_train, y_validation = singapurAnalyser.
 ↪trainingTestSplit(X, y, TEST_SIZE, shuffle)
    model = SVMModel(dataset, users)
    fpr[legalUser], tpr[legalUser], thr = model.evaluate(X_train, y_train,␣
 ↪X_validation, y_validation, user = legalUser)
    threshold = -1
    try:
```

```
        eer = brentq(lambda x: 1. - x - interp1d(fpr[legalUser],␣
↪tpr[legalUser])(x), 0., 1.) ## brentq znajdowanie 0 , interpid interpolacja
        threshold = interp1d(fpr[legalUser], thr)(eer)

    except (ZeroDivisionError, ValueError):
        print("Division by zero")

    roc_auc[legalUser] = auc(fpr[legalUser], tpr[legalUser])
    print(f"{legalUser}: {roc_auc[legalUser]:.3f} threshold: {threshold:.3f}")

plotROCs(fpr, tpr, roc_auc, users)
```

```
User 1 is being analyzed:
CV Accuracy: 0.60 (+/- 0.08)
Test Accuracy: 0.64
1: 0.802 threshold: 0.499
User 2 is being analyzed:
CV Accuracy: 0.65 (+/- 0.07)
Test Accuracy: 0.63
2: 0.823 threshold: 0.484
User 3 is being analyzed:
CV Accuracy: 0.63 (+/- 0.08)
Test Accuracy: 0.64
3: 0.817 threshold: 0.532
User 4 is being analyzed:
CV Accuracy: 0.62 (+/- 0.08)
Test Accuracy: 0.64
4: 0.842 threshold: 0.476
User 5 is being analyzed:
CV Accuracy: 0.65 (+/- 0.16)
Test Accuracy: 0.65
5: 0.860 threshold: 0.514
User 6 is being analyzed:
CV Accuracy: 0.59 (+/- 0.09)
Test Accuracy: 0.58
6: 0.678 threshold: 0.510
User 9 is being analyzed:
CV Accuracy: 0.59 (+/- 0.08)
Test Accuracy: 0.62
9: 0.851 threshold: 0.515
User 7 is being analyzed:
CV Accuracy: 0.64 (+/- 0.06)
Test Accuracy: 0.65
7: 0.847 threshold: 0.501
User 16 is being analyzed:
CV Accuracy: 0.89 (+/- 0.02)
Test Accuracy: 0.89
16: 0.996 threshold: 0.453
```

```
User 20 is being analyzed:
CV Accuracy: 0.62 (+/- 0.05)
Test Accuracy: 0.66
20: 0.863 threshold: 0.508
User 21 is being analyzed:
CV Accuracy: 0.69 (+/- 0.06)
Test Accuracy: 0.70
21: 0.913 threshold: 0.495
User 23 is being analyzed:
CV Accuracy: 0.62 (+/- 0.10)
Test Accuracy: 0.62
23: 0.741 threshold: 0.499
```



The function below extract the stolen sessions and save it to the test_files path

```
[ ]: #singapur_reader.processTestData()
```

9 DFL

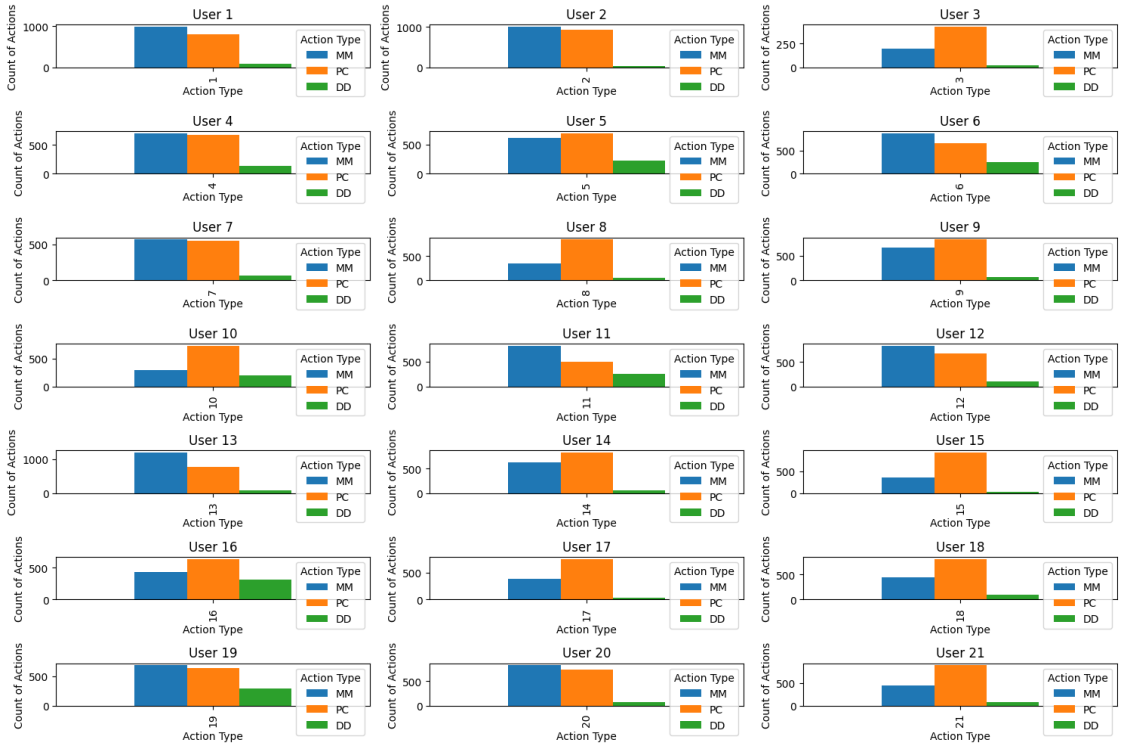
[ ]:
```
from utils.consts import *
from utils. plotting import *
from data_reader_dfl import *
from nonlegality_analyser import *
import random
```

[ ]:
```
users = [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16, 17, 18, 19, 20, 21]
dfl_reader = DataReaderDfl(DFL, users, False, limit = 1000)
dfl_reader.processDataWithoutLabels()
```

[ ]:
```
path = dfl_reader.getFileName()
dflAnalyser = nonLegalityAnalyser(path)
print(dflAnalyser.countActions())
```

```
type_of_action     1    3    4
userid
1                984  795   78
2               1004  929   37
3                196  426   22
4                702  678  140
5                611  692  226
6                882  658  256
7                569  555   63
8                341  839   51
9                666  831   70
10               299  720  203
11               812  493  260
12               826  682  109
13              1204  784   71
14               635  831   51
15               366  935   29
16               424  636  313
17               383  760   39
18               449  811   88
19               680  629  285
20               825  731   77
21               443  885   83
```

[ ]:
```
dflAnalyser.plotActionHistograms()
```



9.1 Random Forest

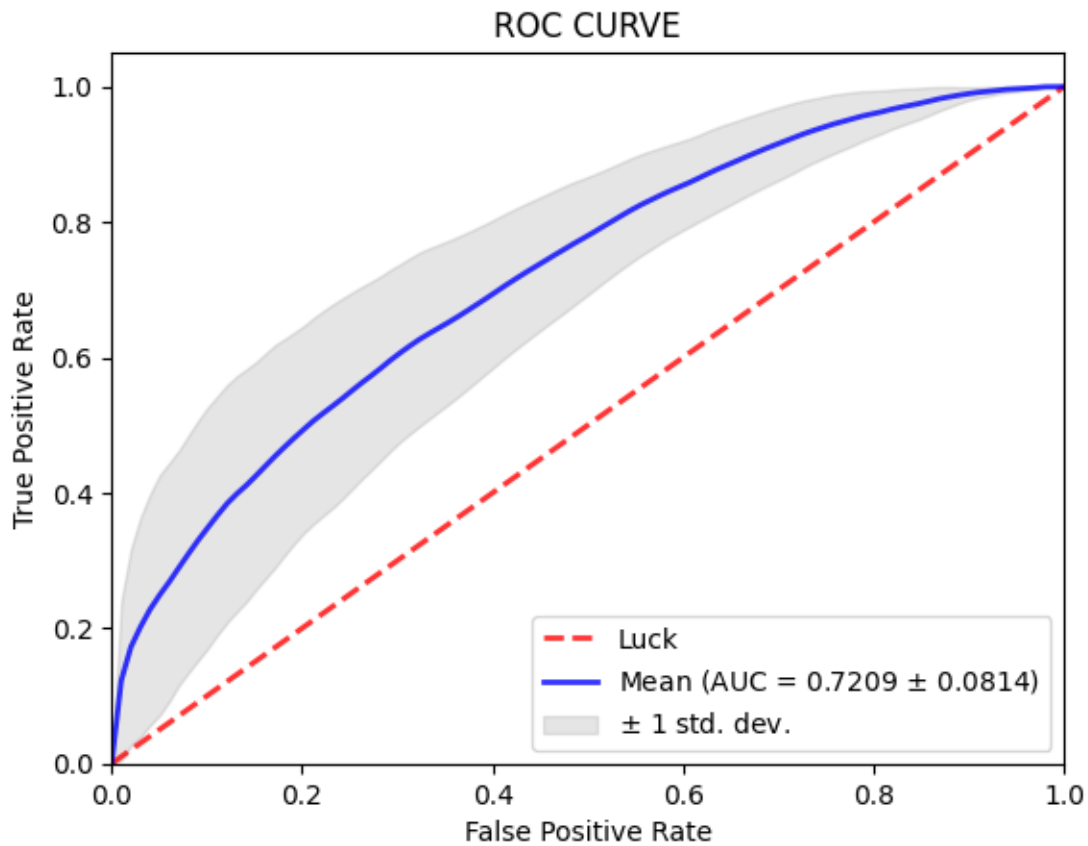
[ ]:
```
from RandomForestModel import *
```

[ ]:
```
shuffle = True
fpr = {}
tpr = {}
roc_auc = {}
for legalUser in users:
    dataset = dflAnalyser.createTrainingDataWithLabel(legalUser)
    X = dataset[:, 0:-1]
    y = dataset[:, -1]
    X_train, X_validation, y_train, y_validation = dflAnalyser.
 ↪trainingTestSplit(X, y, TEST_SIZE, shuffle)
    model = RandomForestModel(dataset, users)
    fpr[legalUser], tpr[legalUser], thr = model.evaluate(X_train, y_train,␣
 ↪X_validation, y_validation, scale = True, user = legalUser)
    threshold = -1
    try:
        eer = brentq(lambda x: 1. - x - interp1d(fpr[legalUser],␣
 ↪tpr[legalUser])(x), 0., 1.) ## brentq znajdowanie 0 , interpid interpolacja
        threshold = interp1d(fpr[legalUser], thr)(eer)
```

```
    except (ZeroDivisionError, ValueError):
        print("Division by zero")

    roc_auc[legalUser] = auc(fpr[legalUser], tpr[legalUser])
    print(f"{legalUser}: {roc_auc[legalUser]:.3f} threshold: {threshold:.3f}")

plotROCs(fpr, tpr, roc_auc, users)
```

```
User 1 is being analyzed:
CV Accuracy: 0.73 (+/- 0.07)
Test Accuracy: 0.52
1: 0.650 threshold: 0.378
User 2 is being analyzed:
CV Accuracy: 0.79 (+/- 0.04)
Test Accuracy: 0.56
2: 0.676 threshold: 0.412
User 3 is being analyzed:
CV Accuracy: 0.79 (+/- 0.09)
Test Accuracy: 0.71
3: 0.760 threshold: 0.474
User 4 is being analyzed:
CV Accuracy: 0.93 (+/- 0.05)
Test Accuracy: 0.76
4: 0.934 threshold: 0.351
User 5 is being analyzed:
CV Accuracy: 0.70 (+/- 0.06)
Test Accuracy: 0.52
5: 0.571 threshold: 0.384
User 6 is being analyzed:
CV Accuracy: 0.76 (+/- 0.03)
Test Accuracy: 0.58
6: 0.679 threshold: 0.438
User 7 is being analyzed:
CV Accuracy: 0.85 (+/- 0.06)
Test Accuracy: 0.69
7: 0.766 threshold: 0.516
User 8 is being analyzed:
CV Accuracy: 0.85 (+/- 0.05)
Test Accuracy: 0.55
8: 0.606 threshold: 0.252
User 9 is being analyzed:
CV Accuracy: 0.80 (+/- 0.03)
Test Accuracy: 0.56
9: 0.720 threshold: 0.445
User 10 is being analyzed:
CV Accuracy: 0.89 (+/- 0.03)
Test Accuracy: 0.48
10: 0.685 threshold: 0.374
```

```
User 11 is being analyzed:
CV Accuracy: 0.78 (+/- 0.04)
Test Accuracy: 0.67
11: 0.768 threshold: 0.502
User 12 is being analyzed:
CV Accuracy: 0.76 (+/- 0.03)
Test Accuracy: 0.54
12: 0.674 threshold: 0.377
User 13 is being analyzed:
CV Accuracy: 0.76 (+/- 0.04)
Test Accuracy: 0.55
13: 0.722 threshold: 0.405
User 14 is being analyzed:
CV Accuracy: 0.72 (+/- 0.05)
Test Accuracy: 0.52
14: 0.680 threshold: 0.405
User 15 is being analyzed:
CV Accuracy: 0.88 (+/- 0.04)
Test Accuracy: 0.76
15: 0.850 threshold: 0.500
User 16 is being analyzed:
CV Accuracy: 0.82 (+/- 0.05)
Test Accuracy: 0.68
16: 0.736 threshold: 0.484
User 17 is being analyzed:
CV Accuracy: 0.85 (+/- 0.05)
Test Accuracy: 0.64
17: 0.817 threshold: 0.560
User 18 is being analyzed:
CV Accuracy: 0.87 (+/- 0.04)
Test Accuracy: 0.71
18: 0.764 threshold: 0.494
User 19 is being analyzed:
CV Accuracy: 0.72 (+/- 0.08)
Test Accuracy: 0.52
19: 0.629 threshold: 0.384
User 20 is being analyzed:
CV Accuracy: 0.81 (+/- 0.04)
Test Accuracy: 0.64
20: 0.700 threshold: 0.511
User 21 is being analyzed:
CV Accuracy: 0.76 (+/- 0.06)
Test Accuracy: 0.66
21: 0.755 threshold: 0.465
```



9.2 SVM

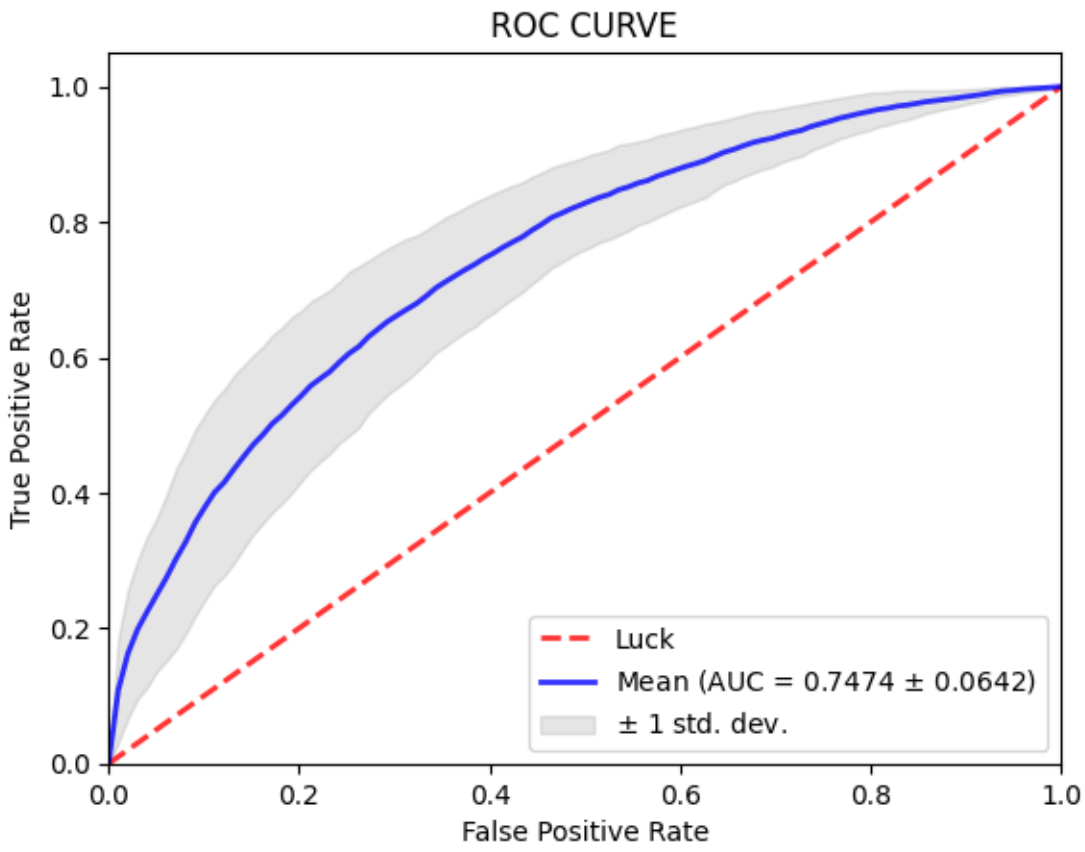
[ ]:
```
from SVMmodel import *
```

[ ]:
```
shuffle = True
fpr = {}
tpr = {}
roc_auc = {}
for legalUser in users:
    dataset = dflAnalyser.createTrainingDataWithLabel(legalUser)
    X = dataset[:, 0:-1]
    y = dataset[:, -1]
    X_train, X_validation, y_train, y_validation = dflAnalyser.
 ↪trainingTestSplit(X, y, TEST_SIZE, shuffle)
    model = SVMModel(dataset, users)
    fpr[legalUser], tpr[legalUser], thr = model.evaluate(X_train, y_train,␣
 ↪X_validation, y_validation, user = legalUser)
    threshold = -1
    try:
```

```
        eer = brentq(lambda x: 1. - x - interp1d(fpr[legalUser],␣
↪tpr[legalUser])(x), 0., 1.) ## brentq znajdowanie 0 , interpid interpolacja
        threshold = interp1d(fpr[legalUser], thr)(eer)

    except (ZeroDivisionError, ValueError):
        print("Division by zero")

    roc_auc[legalUser] = auc(fpr[legalUser], tpr[legalUser])
    print(f"{legalUser}: {roc_auc[legalUser]:.3f} threshold: {threshold:.3f}")

plotROCs(fpr, tpr, roc_auc, users)
```

```
User 1 is being analyzed:
CV Accuracy: 0.65 (+/- 0.04)
Test Accuracy: 0.66
1: 0.707 threshold: 0.520
User 2 is being analyzed:
CV Accuracy: 0.68 (+/- 0.04)
Test Accuracy: 0.70
2: 0.789 threshold: 0.450
User 3 is being analyzed:
CV Accuracy: 0.65 (+/- 0.11)
Test Accuracy: 0.68
3: 0.749 threshold: 0.413
User 4 is being analyzed:
CV Accuracy: 0.78 (+/- 0.08)
Test Accuracy: 0.79
4: 0.880 threshold: 0.428
User 5 is being analyzed:
CV Accuracy: 0.56 (+/- 0.07)
Test Accuracy: 0.58
5: 0.606 threshold: 0.508
User 6 is being analyzed:
CV Accuracy: 0.63 (+/- 0.05)
Test Accuracy: 0.62
6: 0.722 threshold: 0.552
User 7 is being analyzed:
CV Accuracy: 0.70 (+/- 0.05)
Test Accuracy: 0.69
7: 0.782 threshold: 0.429
User 8 is being analyzed:
CV Accuracy: 0.61 (+/- 0.09)
Test Accuracy: 0.66
8: 0.724 threshold: 0.502
User 9 is being analyzed:
CV Accuracy: 0.70 (+/- 0.05)
Test Accuracy: 0.70
9: 0.765 threshold: 0.545
```

```
User 10 is being analyzed:
CV Accuracy: 0.72 (+/- 0.08)
Test Accuracy: 0.76
10: 0.834 threshold: 0.597
User 11 is being analyzed:
CV Accuracy: 0.66 (+/- 0.06)
Test Accuracy: 0.64
11: 0.714 threshold: 0.470
User 12 is being analyzed:
CV Accuracy: 0.66 (+/- 0.05)
Test Accuracy: 0.69
12: 0.745 threshold: 0.510
User 13 is being analyzed:
CV Accuracy: 0.64 (+/- 0.05)
Test Accuracy: 0.63
13: 0.683 threshold: 0.482
User 14 is being analyzed:
CV Accuracy: 0.65 (+/- 0.06)
Test Accuracy: 0.67
14: 0.761 threshold: 0.543
User 15 is being analyzed:
CV Accuracy: 0.81 (+/- 0.05)
Test Accuracy: 0.82
15: 0.884 threshold: 0.456
User 16 is being analyzed:
CV Accuracy: 0.68 (+/- 0.06)
Test Accuracy: 0.66
16: 0.721 threshold: 0.506
User 17 is being analyzed:
CV Accuracy: 0.66 (+/- 0.08)
Test Accuracy: 0.68
17: 0.768 threshold: 0.423
User 18 is being analyzed:
CV Accuracy: 0.71 (+/- 0.05)
Test Accuracy: 0.71
18: 0.756 threshold: 0.455
User 19 is being analyzed:
CV Accuracy: 0.63 (+/- 0.05)
Test Accuracy: 0.63
19: 0.681 threshold: 0.545
User 20 is being analyzed:
CV Accuracy: 0.62 (+/- 0.04)
Test Accuracy: 0.61
20: 0.672 threshold: 0.490
User 21 is being analyzed:
CV Accuracy: 0.66 (+/- 0.04)
Test Accuracy: 0.67
21: 0.758 threshold: 0.389
```


ROC CURVE
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
Luck
Mean (AUC = 0.7474 ± 0.0642)
± 1 std. dev.